

遼寧師範大學

Liaoning Normal University

（单击输入 4 位年，如 2010 届）

本 科 生 毕 业 论 文（设 计）

题　　目：　单击输入论文题目

学　　院：　单击输入学院全称

专　　业：　单击输入规范的专业名称

班级序号：　单击输入如 5 班 9 号

学　　号：　单击输入 14 位学号

学生姓名：　单击输入学生姓名

指导教师：　单击输入教师姓名

单击输入 4 位年，如 2010 年 5 月

辽宁师范大学

本科生毕业论文（设计）原创性声明

本人郑重声明：所呈交的毕业论文（设计）是本人在导师指导下独立进行研究工作所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

毕业论文（设计）作者签名：_____

日期：　　年　　月　　日

目　　录

摘　　要 1
Abstract 2
前　　言 3
第一章　绪论 4
1.1 研究背景与研究意义 4
1.1.1 背景 4
1.1.2 研究问题 4
1.1.3 研究意义 4
1.2 国内外研究现状 5
1.2.1 LSM-Tree 与键值存储研究 5
1.2.2 冷热分层与异构存储研究 6
1.2.3 分布式键值存储与一致性机制 6
1.2.4 云原生数据库与控制平面研究 7
1.2.5 系统可观测性与运行管理研究 7
1.2.6 国内外研究现状评述 7
1.3 本文研究内容 8
1.4 论文组织结构 9
本章引用文献 9
第二章　相关技术与系统总体设计 10
2.1 系统需求与设计目标 10
2.1.1 需求来源 10
2.1.2 设计目标 10
2.1.3 非目标 11
2.2 相关核心技术 11
2.2.1 Java 21 与 JVM 11
2.2.2 Netty 与 RESP 协议 11
2.2.3 LSM-Tree 11
2.2.4 Raft 与混合逻辑时钟 11
2.2.5 Spring Boot 与控制平面 12
2.3 存储引擎相关技术 12
2.3.1 预写日志（WAL） 12
2.3.2 内存表（MemTable） 12
2.3.3 不可变磁盘表（SSTable）与布隆过滤器 12
2.3.4 合并（Compaction） 12
2.4 冷热分层与缓存技术 12
2.4.1 分层的基本思路 12
2.4.2 内存映射（mmap） 13
2.4.3 块缓存策略：LRU 而非 ARC 13
2.4.4 异步迁移与背压 13
2.5 分布式与一致性技术 13
2.5.1 分片 13
2.5.2 Raft 复制 13

2.5.3 时间戳与两阶段提交 13
2.5.4 MVCC 的基本概念与本文边界 14
2.6 系统总体架构设计 14
2.6.1 三个层面 14
2.6.2 边界与数据流 15
2.6.3 核心技术选型 15
2.7 本章小结 15
本章引用文献 16
第三章 系统核心存储引擎设计与实现 17
3.1 设计目标与问题背景 17
3.1.1 为什么自己实现 LSM 主链路 17
3.1.2 设计目标 17
3.1.3 非目标 17
3.2 总体方案与模块划分 17
3.2.1 分层结构 18
3.2.2 依赖方向与装饰器 18
3.2.3 关键选型与理由 19
3.3 WAL：顺序持久化与恢复 19
3.3.1 设计目标 19
3.3.2 结构 19
3.3.3 关键实现：空闲期也必须落盘 19
3.3.4 恢复流程 20
3.3.5 局限 20
3.4 MemTable：内存有序表与内存记账 20
3.4.1 设计目标 20
3.4.2 结构 20
3.4.3 内存记账与水位 20
3.4.4 TTL 的处理 20
3.4.5 局限 21
3.5 SSTable：不可变磁盘表 21
3.5.1 设计目标 21
3.5.2 文件布局 21
3.5.3 写与读 21
3.5.4 元数据与键范围 21
3.5.5 局限 21
3.6 Compaction：合并与空间回收 22
3.6.1 设计目标 22
3.6.2 触发与执行 22
3.6.3 并发安全 22
3.6.4 实测 22
3.6.5 局限 22
3.7 冷热分层：水位、迁移与缓存 22
3.7.1 设计目标 22
3.7.2 触发链路 22

3.7.3 读取回源 23
3.7.4 BlockCache 23
3.7.5 局限 23
3.8 核心模块类图（图 3-2） 23
3.9 实验与结果 24
3.9.1 环境 24
3.9.2 与 Redis 7 的横向对比 24
3.9.3 分层调度 25
3.9.4 持久化边界 25
3.9.5 存储结构与清单成本 25
3.9.6 小结 25
3.10 本章局限 26
本章引用文献 26
第四章　系统控制与管理模块设计与实现 27
4.1 设计目标与问题背景 27
4.1.1 为什么需要独立的控制面 27
4.1.2 设计目标 27
4.1.3 非目标 27
4.2 RESP 协议与 Netty 接入 27
4.2.1 协议实现 27
4.2.2 Netty 接入与并发模型 28
4.2.3 命令层与扩展 28
4.2.4 局限 28
4.3 命令执行与存储引擎的边界 28
4.3.1 接口收敛 28
4.3.2 事务与集群的接入点 28
4.3.3 局限 28
4.4 INFO 可观测性：从内核计数到界面数字 28
4.4.1 设计目标 29
4.4.2 数据来源分层 29
4.4.3 请求时序 29
4.4.4 延迟分位 29
4.4.5 实测开销 30
4.4.6 局限 30
4.5 控制面：认证、审计与多实例 30
4.5.1 认证与授权 30
4.5.2 审计 30
4.5.3 多实例隔离 30
4.5.4 失败语义 30
4.5.5 局限 31
4.6 备份与恢复 31
4.6.1 设计目标 31
4.6.2 时序 31
4.6.3 三个关键决策 32

4.6.4 实验 32
4.6.5 局限 32
4.7 本章实验与小结 32
4.7.1 端到端验收 32
4.7.2 故障注入 33
4.7.3 小结 33
4.8 本章局限 33
本章引用文献 33
第五章 分布式架构与一致性机制设计 34
5.0 证据分级说明（本章的阅读前提） 34
5.1 设计目标与总体架构 34
5.1.1 目标 34
5.1.2 分层 34
5.1.3 非目标 35
5.2 集群管理与数据分片 35
5.2.1 两级分片 35
5.2.2 元数据 36
5.2.3 Region 生命周期 36
5.3 Raft 共识与日志复制 36
5.3.1 实现范围 36
5.3.2 提交规则 37
5.3.3 与存储层的接口 37
5.3.4 局限 37
5.4 故障检测、领导者选举与恢复 37
5.4.1 检测 37
5.4.2 选举与脑裂防护 37
5.4.3 恢复路径 37
5.4.4 局限 38
5.5 MVCC 与混合逻辑时钟（HLC） 38
5.5.1 为什么需要 HLC 38
5.5.2 实现 38
5.5.3 MVCC 的实际范围 38
5.5.4 局限 38
5.6 基于 Percolator 思想的分布式事务协调 38
5.6.1 协议 39
5.6.2 实现组件 39
5.6.3 决策顺序 39
5.6.4 与 Percolator 的差异（必须写明） 39
5.6.5 局限 40
5.7 数据迁移与一致性保障 40
5.8 测试验证与局限性分析 40
5.8.1 测试规模 40
5.8.2 已验证的结论 40
5.8.3 未验证的结论（不得写成已实现） 40

5.8.4 小结 41
本章引用文献 41
第六章 系统测试与实验结果分析 42
6.1 测试目标与测试策略 42
6.1.1 测试要回答的问题 42
6.1.2 分层策略 42
6.2 测试环境与配置 42
6.3 核心功能测试 43
6.3.1 存储与分层 43
6.3.2 集群与事务 43
6.3.3 控制台 43
6.4 端到端业务闭环测试 43
6.4.1 七条闭环与断言 44
6.4.2 断言的写法 44
6.5 故障注入与恢复测试 44
6.6 性能测试与分析 45
6.6.1 口径先于数字 45
6.6.2 与 Redis 7 的对比 46
6.6.3 观测开销 46
6.6.4 持久化边界 46
6.6.5 合并与清单成本 46
6.7 工程质量与覆盖率 47
6.7.1 关于 13 项跳过 47
6.7.2 覆盖率的正确解读 47
6.8 缺陷发现与修复验证 47
6.8.1 缺陷矩阵 47
6.8.2 案例一：TD-103（WAL 持久化） 48
6.8.3 案例二：TD-101（备份并发） 48
6.8.4 案例三：TD-105（数据库故障恢复） 48
6.9 测试局限性分析 49
6.10 本章小结 49
本章引用文献 49
第七章 总结与展望 50
7.1 全文工作总结 50
7.2 主要工作与工程成果 50
7.3 系统局限性 51
7.4 后续研究与改进方向 52
7.5 本章小结 52
参考文献 53

摘　要

内存键值数据库以低延迟支撑了大量在线业务，但其容量受物理内存约束：当数据集增长到数十 GB 以上时，内存成本成为瓶颈，而其中真正被频繁访问的往往只是一小部分热数据。针对这一问题，本文设计并实现了一个 Redis 协议兼容、面向冷热数据分层的键值存储引擎，以及与之配套的独立 Web 运维控制台。

系统分为数据平面与控制平面两个进程。数据平面自行实现 LSM 主链路：分段预写日志、分片有序内存表、带块索引与布隆过滤器的不可变磁盘表，以及按表数阈值触发的合并；在此之上以水位线驱动落盘、异步迁移与背压，使内存占用成为可配置的硬约束，冷数据在读取时按需回源并写回热层。控制平面通过标准 RESP 协议纳管内核，提供认证授权、指标看板、键空间浏览、在线终端、告警、备份恢复与审计，其观测数据全部来自内核计数，并对"请求失败、内核未上报、真实为零"三种状态分别呈现。

验证方面，本文建立了单元与集成测试、端到端业务闭环、故障注入、性能基准与覆盖率五个层次的证据链。内核全量回归执行 15014 次、零失败；控制平面 93 例测试、行覆盖率 62.57%；七条业务闭环共 91 条断言、四类故障注入共 22 条断言全部通过。验证过程中发现并修复了预写日志空闲期不落盘、同实例并发备份双双成功、失败操作审计被事务回滚等真实缺陷，其中前者的崩溃丢失窗口由"任意量"收敛为实测 420.8 毫秒。

实验表明：在承担预写日志持久化成本的前提下，系统吞吐达到同机 Redis 的 0.36 至 0.47 倍；分层调度在十万至百万条目规模下迁移成功率 100%；观测开销在 8 张表规模下对业务延迟无可测影响。本文同时明确标注了尚未验证的边界：分布式能力仅在单机多进程环境验证，时间戳机制不等于完整的多版本并发控制，进程崩溃恢复不等于掉电安全。

关键词：键值存储；冷热分层；LSM-Tree；可观测性；工程验证

Abstract

In-memory key-value databases support a large number of online services with low latency, but their capacity is bounded by physical memory. When the data set grows beyond tens of gigabytes, memory cost becomes the bottleneck, while only a small fraction of the data is frequently accessed. This thesis designs and implements a Redis-protocol-compatible key-value storage engine with hot/cold data tiering, together with an independent web-based operations console.

The system is split into a data plane and a control plane. The data plane implements the LSM pipeline from scratch: a segmented write-ahead log, a sharded ordered memtable, immutable SSTables with block index and Bloom filter, and compaction triggered by table count. On top of it, watermark-driven flush, asynchronous migration and back pressure turn memory usage into a configurable hard constraint, and cold reads are served on demand and copied back to the hot tier. The control plane manages engine instances over the standard RESP protocol and provides authentication, metrics, keyspace browsing, an online terminal, alerting, backup and restore, and auditing. All observation data come from engine counters, and three states — request failure, metric not reported, and a genuine zero — are presented separately.

Verification follows a five-layer evidence chain: unit and integration tests, end-to-end business loops, fault injection, performance benchmarks, and coverage. The full kernel regression executed 15014 tests with zero failures; the control plane has 93 tests with 62.57% line coverage; seven business loops contribute 91 assertions and four fault-injection scenarios contribute 22 assertions, all passing. Real defects were found and fixed, including a write-ahead log that never flushed while idle, concurrent duplicate backups on the same instance, and audit records rolled back together with failed operations. The crash loss window of the first defect was reduced from unbounded to a measured 420.8 milliseconds.

Experiments show that, while carrying the cost of write-ahead logging, throughput reaches 0.36 to 0.47 times that of Redis on the same host; tiering migration achieves a 100% success rate from one hundred thousand to one million entries; and observation overhead has no measurable impact on latency at the eight-table scale. The thesis also states explicit boundaries: distributed capabilities are verified only in a single-machine multi-process setting, the timestamp mechanism is not full multi-version concurrency control, and process-crash recovery is not power-failure safety.

Key words: key-value store; hot/cold tiering; LSM-Tree; observability; engineering verification

前　言

本文的研究对象是一个面向冷热数据分层的键值存储系统及其配套的运维控制台。系统以 Redis 协议为对外接口，以 LSM 结构为存储主链路，在单机内存受限的条件下，通过水位驱动的落盘、迁移与回读机制承载超出内存容量的数据，并对外暴露可核验的运行指标。

本文的工作范围包括四个部分：其一是存储引擎与冷热分层机制的实现；其二是独立控制平面与可观测性体系的实现；其三是在有限范围内讨论分布式复制与一致性机制；其四是建立一套可复现的工程验证方法，并对每项能力标注其证据边界。系统的分布式能力在单机多进程环境下验证，未覆盖真实跨机网络与掉电场景，相关结论的适用范围在各章末尾与第七章中明确说明。

论文的组织如下：第二章介绍相关技术与总体设计；第三章至第五章分别阐述存储引擎、控制与管理模块、分布式与一致性机制的设计与实现；第六章给出测试与实验结果并分析其边界；第七章总结全文并讨论后续工作。各章的设计描述与其验证结果相互对应，文中出现的每一个实验数字均可追溯到具体的测试或测量过程。

第一章　绪论

论文正文初稿 • 对应基线 425f4c6

图表：图 1-1（研究脉络与本文定位）

文献：15 篇核心（见 chapter1-reference-map.md），技术规范单列

1.1 研究背景与研究意义

1.1.1 背景

键值存储是互联网系统最基础的组件之一。以 Redis 为代表的内存键值数据库凭借丰富的数据结构与亚毫秒级响应，

长期承担缓存、会话、计数器与排行榜等职责。但它的容量模型有一个硬约束：**数据必须整体驻留在内存中**。

当数据集从 GB 级增长到数十 GB 甚至更大时，内存成本与单机容量同时成为瓶颈，而其中真正被频繁访问的往往只是一小部分"热数据"。

另一条技术路线是持久化存储引擎。LSM-Tree 通过"内存有序结构 + 磁盘顺序写 + 后台合并"，

把随机写转化为顺序写，在写入密集的场景下显著优于传统的磁盘索引结构 [1]；

Bigtable 则给出了这一思想在工业系统中的工程形态——内存表、不可变文件与合并的三段式结构 [2]。

在此基础上，"把冷数据放到慢速介质、把热数据留在内存"的分层存储成为自然延伸：

近年来的研究进一步关注**热度识别与记录提升**，即写入热点与读取热点可能并不在同一层级 [3]，

以及本地介质与远端存储组合时的放置、缓存与恢复问题 [4]。

与此同时，数据库系统的部署形态也在变化。云原生数据库把计算与存储解耦，用弹性资源与副本一致性支撑多租户场景 [5]；

数据中心资源解耦后，数据组织方式本身也需要重新设计 [6]。这使得"管理面"从附属工具变成一等公民：

系统的可观测性、可运维性与可验证性，直接决定了它能否被真正使用。

1.1.2 研究问题

上述三条线索交汇处存在一个工程问题：**能否在一个协议兼容的键值引擎中，同时做到"内存受配额约束"、

"冷数据可按需回源"与"运行状态可被外部验证"？**

这个问题的难点不在单点技术，而在三者的耦合：分层会引入新的状态（谁在内存、谁在磁盘、迁移进行到哪一步），

这些状态必须被一致地观测；而观测本身又会带来开销，需要在指标完整性与业务延迟之间取舍。

1.1.3 研究意义

本文的意义体现在两个层面。

工程层面：把 Redis 协议兼容、LSM 存储、冷热分层与独立控制面整合为一套可运行、可观测、可运维的系统，

并给出每条能力"被证明到什么程度"的明确边界。对于教学与中小规模部署场景，

这种"可解释的完整实现"

比直接调用成熟组件更有参考价值。

方法层面：本文把"可证伪断言"作为验证的核心手段——每条结论都对应一个可以被推翻的检查点，

并用故障注入去主动制造失败。实践证明，这套方法发现的不是语法错误，而是并发一致性、持久化语义与审计完整性上的真实缺陷。



图 1-1 研究脉络与本文定位（工程整合与验证，而非提出新的存储理论）

如图 1-1 所示，本文不试图提出新的存储理论，而是站在既有研究之上做**工程整合与验证**：

把已被验证的机制组合成一个协议兼容、可观测、可验证的系统，并如实标注每项能力的证据等级。

1.2 国内外研究现状

1.2.1 LSM-Tree 与键值存储研究

LSM-Tree 的原始动机是降低高频写入下的索引维护成本：把随机更新先写入内存有序结构，

积累到阈值后顺序落盘，再用后台合并把多层文件整理成有序序列 [1]。

Bigtable 把这一思想工程化，明确了"内存表—不可变文件—合并"的职责划分 [2]，

此后绝大多数基于 LSM 的键值存储都沿用了这一骨架。

但 LSM 并非没有代价。围绕它形成了三组此消彼长的放大：**写放大**（同一份数据在合并中被反复重写）、

读放大（一次查询可能要查多层多个文件）与**空间放大**（旧版本与删除标记占用空间）。

后续研究大体沿着"在某一个放大上换取代价"展开：

- Monkey 研究布隆过滤器的内存分配，指出过滤器占用的内存与读放大之间存在最优分配点 [7]；
- Dostoevsky 进一步讨论合并频率的选择，说明"合并得多"会降低读与空间放大但抬高写放大，
  并提出根据负载自适应地减少"多余合并" [8]；
- 国内综述则从工程实现角度梳理了 LSM 键值存储的关键技术与优化方向 [9]。

比较与评述：这一方向的研究已经相当充分，但结论高度依赖负载假设。

对于以内存命中的热读为主、写入相对均匀的场景，"按表数阈值触发合并"这类简化策略

虽然牺牲了部分空间回收效率，却换来了行为可预测与实现成本可控——这正是本文采取的策略，

其代价在 3.6.5 中有明确说明。

1.2.2 冷热分层与异构存储研究

分层存储的基本假设是"访问分布不均"：少数热数据承担多数访问，其余数据可以放到更慢、更便宜的介质上。

但要把它做成工程机制，需要回答三个问题：**如何识别热度、何时迁移、迁移期间如何保证一致性**。

HotRAP 指出，在采用分层存储的 LSM 系统中，**写入热点与读取热点可能位于不同层级**：

新写入的数据停留在上层，而真正被频繁读取的记录可能已被下沉，导致读性能与存储效率之间出现矛盾；

其解法是跟踪记录热度并把热点记录提升回上层 [3]。

RocksMash 则把分层扩展到"本地存储 + 云存储"的组合，讨论数据放置、细粒度缓存（LAP cache）与故障恢复 [4]。

比较与评述：两项工作的共同点是把"放置决策"看作动态优化问题，差异在于介质假设——

前者面向 SSD/HDD 的层级，后者面向本地与远端的分层。二者都表明：冷热分层的关键不是"能不能落盘"，

而是"热度判断与迁移时机"。

与它们相比，本文的分层是**水位驱动**的简化机制：按内存使用率触发落盘与迁移，

没有实现基于访问频次的热点提升。这一取舍让系统行为容易解释与测试，但也意味着在访问分布剧烈倾斜时，

热记录可能仍留在冷层（3.7.5 已列为局限）。

1.2.3 分布式键值存储与一致性机制

当单机容量与可用性都不再足够时，系统需要走向多副本。围绕"多副本如何达成一致"，形成了两条代表性路线。

一条是**去中心化的最终一致**路线。Dynamo 用一致性哈希做分片、用向量时钟处理冲突、用可调读写法定人数在

可用性与一致性之间滑动 [10]，其目标是"永远可写"。另一条是**基于共识的强一致**路线：

Raft 把共识分解为领导者选举、日志复制与安全性三个子问题 [11]，

以更易实现与解释的方式达到与 Paxos 同等的安全性，成为近年来分布式存储的主流选择。

跨分片事务则引出时间戳问题。Spanner 用 TrueTime 把"时间不确定性"显式暴露，通过等待不确定区间获得外部一致性 [12]；

Percolator 用"时间戳 + 锁记录"替代阻塞式两阶段提交，在 Bigtable 之上实现跨行事务 [13]；

混合逻辑时钟（HLC）则在无法获得全局物理时钟时，用"物理毫秒 + 逻辑计数"保证时间戳单调可比 [14]。

比较与评述：三种时间戳方案对应不同的假设与代价——TrueTime 需要专用硬件（GPS/原子钟），

TSO 需要集中式授时服务，HLC 只需要松散同步的物理时钟但只能保证单调与因果序。

本文采用"Raft 复制 + HLC/TSO 时间戳 + 两阶段提交"的组合，属于第三类：

代价低、可解释，但**不提供线性一致读**，也未实现 Percolator 的多版本存储（5.5.3、5.6.4 已明确边界）。

1.2.4 云原生数据库与控制平面研究

云原生数据库的核心是**资源解耦**：把计算、存储与日志分离，使三者可以独立扩展与故障恢复。

综述性工作系统梳理了这一形态下的关键技术，包括计算存储分离的架构、副本一致性、弹性资源管理与故障恢复 [5]；

针对解耦数据中心的讨论则进一步指出，当网络成为存储访问的一部分时，数据组织方式（放置、缓存、复制）需要重新设计 [6]。

比较与评述：这类研究关注的往往是"如何让数据库本身更云化"。

而另一个常被忽视的维度是**管理面**：当实例数量增加、部署形态多样时，

纳管、监控、备份与审计本身就成为系统能力的一部分。

本文采取的做法是把控制面做成**独立进程**，只通过协议访问数据面，

从而在不改造存储引擎的前提下获得管理能力（第 4 章）。这一设计与"计算存储分离"的动机一致，

但解耦的是"管理与数据"，而非"计算与存储"。

1.2.5 系统可观测性与运行管理研究

随着系统从单体走向多服务，故障定位的难度从"看日志"变成了"跨服务追踪"。

针对十家企业微服务实践的调查显示，指标与追踪已成为故障定位的基础设施，

而观测数据的价值取决于它能否被**结构化地采集与关联** [15]。

比较与评述：该方向的成果多面向"服务调用链"，而存储系统的观测对象不同——

它关心的是容量、分层、缓存命中、合并进度这些**内部状态**。

两者的共同要求是"指标口径一致、缺失与为零可区分"。

本文据此设计了分层指标与三态展示（4.4），但**没有实现分布式链路追踪**，

因此不引用追踪相关成果作为自身能力的依据。

1.2.6 国内外研究现状评述

把上述五个方向放在一起看，可以看到一条清晰的演进链（图 1-1）：

存储结构从"降低写放大"出发，逐步引入分层与异构介质；

分布式能力从"可用性优先"走向"可验证的强一致"；

部署形态从单机走向解耦，管理面随之独立；

而观测能力则从"看指标"走向"可验证的指标体系"。

这些方向之间并非彼此独立，而是相互牵引：

1. **分层把存储问题变成了状态问题**。一旦数据分布在多个层级，系统就必须维护"谁在哪里、迁移到哪一步"的状态，
   这直接要求更强的观测能力 ^{[6][7]}；
2. **分布式把状态问题变成了时序问题**。多副本与跨分片事务需要全局可比的时间戳，而时间戳方案的选择又会反过来限制一致性等级 ^{[8][10][12]}；
3. **云原生把时序问题变成了管理问题**。资源解耦后，实例数量与形态增加，
   纳管、审计与故障恢复必须由独立的管理面承担 ^{[13][14]}；
4. **可观测性贯穿以上三层**。但现有研究多面向服务调用链 [15]，
   对"存储内部分层状态"的指标体系讨论较少。

由此可以归纳出三点工程挑战，它们构成本文的切入点：

- **协议兼容与分层的耦合**：分层机制必须在既有协议语义下工作，不能要求客户端改造；
- **内部状态的可验证性**：瞬时值、累计值与缺失值必须可区分，否则观测数据无法用于判断；
- **验证成本**：崩溃、掉电与跨机故障难以在普通实验环境中复现，结论必须标注证据边界。

需要强调的是，本文**不主张**"国内外尚无类似系统"或"首次提出分层键值引擎"。
HotRAP 等工作已经表明 LSM 与分层存储的结合是活跃研究方向 [3]，
本文的定位是：面向 **Redis 协议兼容**的键值引擎，探索冷热分层与分布式管理能力的**系统设计、工程实现与验证方法**。

1.3 本文研究内容

围绕上述挑战，本文完成四项工作，每一项对应一个章节，并与第六章的验证结果一一对应。

（1）可解释的 LSM 存储内核与冷热分层机制（第三章）。

针对"内存装不下"的问题，实现预写日志、分片内存表、带块索引与布隆过滤器的不可变表、按阈值触发的合并，
并在此之上用水位线驱动落盘、迁移与背压。研究重点是**让分层行为可解释、可观测**：
内存用量由统一记账维护，分层状态通过 `INFO` 暴露给外部。

（2）独立的控制平面与可观测性体系（第四章）。

针对"内部状态不可验证"的问题，实现以标准协议纳管的控制面，提供认证授权、指标采集、键空间、终端、告警、备份与审计；
并确立三条规则：指标只来自内核计数、缺失与为零必须可区分、失败状态必须有明确错误码。

（3）受限范围的分布式与一致性能力（第五章）。

针对"多副本与跨分片事务"，实现 Raft 日志复制、Region 分片与基于时间戳的两阶段事务，
并明确其证据边界：**在单机多进程环境下验证**，不声称生产级跨机共识。

（4）以可证伪断言为核心的工程验证方法（第六章）。

针对"结论无法追溯"的问题，建立单元/集成测试、端到端业务闭环、故障注入、性能基准与覆盖率五个层次，
并用这套方法发现并修复了持久化、并发一致性与审计完整性上的真实缺陷。

1.4 论文组织结构

全文共七章，组织如下：

章	内容	与其他章的关系
第一章	绪论：研究背景、现状评述、研究内容与结构	提出研究问题
第二章	相关技术与系统总体设计：需求、技术选型、总体架构	回答"用了什么、为什么"
第三章	核心存储引擎设计与实现：WAL、MemTable、SSTable、合并、分层、缓存	回答"存储如何实现"
第四章	控制与管理模块设计与实现：协议接入、可观测性、认证审计、备份恢复	回答"管理与观测如何实现"
第五章	分布式架构与一致性机制：分片、Raft、时间戳、两阶段事务、迁移	回答"分布式能力到什么程度"
第六章	系统测试与实验结果分析：策略、环境、闭环、故障注入、性能、覆盖率、缺陷案例	回答"是否有效、边界在哪"
第七章	总结与展望：工作总结、成果、局限、后续方向	收束全文

各章之间遵循"提出问题—确定技术底座—实现—验证—总结"的逻辑：

第二至第五章的每一项设计，都能在第六章找到对应的验证结果；第六章的每一组数据，都能追溯到具体测试或实验。

本章引用文献

本章引用 [1]–[15]，完整条目见文末《参考文献》（`references.md`）。

技术规范 S1（RESP）、S2（Netty）见文末"技术规范"清单。

第二章　相关技术与系统总体设计

论文正文初稿 • 基线 6439ecf

图表：图 2-1（系统总体架构）、表 2-1（需求与目标映射）、表 2-2（核心技术选型）

基线：6439ecf

本章定位：回答"采用了哪些技术、为什么采用、系统如何组织"；具体实现见第三至第五章。

2.1 系统需求与设计目标

2.1.1 需求来源

系统的需求来自一个具体的使用场景：在单机资源有限的环境中，承载明显超出内存容量的键值数据，

同时保持 Redis 生态的可用性，并能被运维人员观测与管理。由此归纳出四类需求。

功能需求：支持 Redis 常用命令族（字符串、哈希、列表、集合、有序集合、流、地理、位图等），

提供多模型值（JSON、时序、向量）与 SQL 访问能力；支持多实例纳管、键空间浏览、在线终端、

告警、备份恢复与审计。

性能需求：写入路径不因持久化而不可用；读取热数据应有内存级延迟；

冷数据回源应可预测且可观测；分层调度不应阻塞客户端线程。

可靠性需求：进程异常终止后，已落盘的数据可恢复；多副本场景下写入经共识后生效；

控制面在依赖故障（数据库不可用）时能给出明确错误并在依赖恢复后自愈。

管理需求：权限分级、操作可追溯、指标可验证、失败状态可区分（而非以 0 掩盖）。

2.1.2 设计目标

表 2-1　需求、设计目标与对应章节

需求	设计目标	主要落实章节
功能完整	R1 协议兼容：RESP2/RESP3，可用标准客户端直连	4.2
功能完整	R2 数据模型：覆盖常用命令族与多模型值	4.2
性能	R3 写路径可用：持久化不阻塞客户端线程	3.3、3.7
性能	R4 分层有效：内存占用受配额约束，冷数据按需回源	3.7
可靠性	R5 崩溃可恢复：已落盘数据可重放，丢失窗口可量化	3.3
可靠性	R6 复制一致：写经共识生效，单节点故障不丢已提交数据	5.3、5.4
管理	R7 可观测：指标来自内核计数，缺失与为零可区分	4.4
管理	R8 可运维：实例、告警、备份、	4.5、4.6

审计、权限齐备

2.1.3 非目标

- 不与 Redis 比吞吐：本项目承担预写日志的持久化成本，Redis 对比基准关闭了 RDB/AOF；
- 不提供跨机强一致的生产级共识：现有集群能力在单机多进程下验证；
- 不实现完整的多版本并发控制：时间戳用于事务与锁判定，存储层不保留多版本。

2.2 相关核心技术

2.2.1 Java 21 与 JVM

选择 JVM 而非 C/C++ 的理由是**工程可验证性**：JVM 提供成熟的并发原语、内存可见性语义与

可观测工具（JFR、堆转储），便于在论文中复现实验与解释现象；JDK 21 的虚拟线程与

`FileChannel` 异步接口也为后续扩展留出空间。代价是 GC 抖动与内存开销，

因此系统把"内存记账"做成显式机制（见 3.4），而不是依赖堆自动管理。

2.2.2 Netty 与 RESP 协议

RESP 是 Redis 的线协议，分为 RESP2 与 RESP3：前者用 `+ - : $ *` 五种前缀表达简单字符串、错误、

整数、批量字符串与数组；后者增加了 map、set、push 等类型。系统支持两者常用子集，

使 `redis-cli` 与主流 SDK 可以直接连接。

网络层采用 Netty（事件驱动、零拷贝、可插拔编解码），原因是需要**在单连接上保证响应有序**：

命令按 key 分发到多个执行器并行处理，若直接写回会乱序，因此引入序号与顺序写回机制（见 4.2.2）。

2.2.3 LSM-Tree

LSM-Tree 的核心是把随机写转化为顺序写：写入先进入内存结构，积累到阈值后顺序落盘，

后台合并多层以控制读放大（O'Neil 等，1996）[1]。它牺牲部分读性能换取写吞吐与空间利用率，

与本项目"内存装不下、写入频繁、读取以热数据为主"的负载特征吻合，因此选作存储主结构。

2.2.4 Raft 与混合逻辑时钟

Raft 用"领导者选举 + 日志复制"把一致性问题分解为三个子问题（领导者选举、日志复制、安全性），

比 Paxos 更易实现与解释（Ongaro 与 Ousterhout，2014）[11]。本项目以 Raft 作为多副本写入的共识层。

混合逻辑时钟（HLC）把物理时间与逻辑计数结合：物理时间前进时取物理值，回拨时用逻辑计数保证单调，

从而在分布式事务中得到可比的时间戳（Kulkarni 等，2014）[14]。

边界说明：HLC 只解决"时间戳单调可比"，**不等于**多版本并发控制；系统没有实现

多版本存储与垃圾回收。

2.2.5 Spring Boot 与控制平面

控制平面选择 Spring Boot，原因是需要快速组织 REST 接口、鉴权、数据访问与定时任务，

并把"管理面"与"数据面"彻底分离：控制面只通过 RESP 访问内核，不共享进程、不直接读冷层文件。

这一边界使两侧可以独立部署与验证，也让故障定位可以二分（内核算错或控制面解析错）。

2.3 存储引擎相关技术

2.3.1 预写日志（WAL）

WAL 的作用是在修改落入内存结构之前，先把"这次修改"变成可重放的事实。

工业系统通常提供三种落盘策略：每次写都同步（ALWAYS）、按周期批量同步（EVERY_SEC）、

仅写操作系统缓冲（NO）。本项目默认采用周期同步，并把"丢失窗口"作为可度量指标而非隐含假设。

关键认知：EVERY_SEC 只有在"有写入时才触发同步"是不够的——空闲期最后一次写入可能长期停留在用户态缓冲。

因此实现上必须有独立的周期性刷出机制（详见 3.3.3 的缺陷与修复）。

2.3.2 内存表（MemTable）

内存表需要同时满足"有序"与"并发安全"。常见方案是跳表（无锁读、段级写锁）与红黑树（全局锁）。

本项目选择分片跳表：范围扫描天然有序，写入按 key 哈希到分片，降低锁竞争。

内存占用由统一记账维护，使水位判定可以 O(1) 读取。

2.3.3 不可变磁盘表（SSTable）与布隆过滤器

SSTable 一旦写出即不可变，内部由数据块、块索引、布隆过滤器与校验组成。

布隆过滤器（Bloom，1970）[16]用少量位数组回答"键一定不存在"，把无效块读挡在磁盘之前；

Bigtable 的工程实践也采用这一组合（Chang 等，2006）[2]。本项目在此基础上加入了块级校验，

使损坏可被检测而非静默返回错误数据。

2.3.4 合并（Compaction）

合并解决两个问题：表数量增长带来的读放大，以及旧版本与删除标记占用的空间。

常见策略有 size-tiered（按大小分层合并）与 leveled（按层归并）。本项目采用按表数阈值触发的

简化策略：实现成本低、行为可预测，代价是空间回收不如 leveled 彻底（见 3.6.5）。

2.4 冷热分层与缓存技术

2.4.1 分层的基本思路

冷热分层把"内存容量"变成可配置的硬约束：热数据驻留内存表，超过水位后下沉到冷层，

读取时按需回源并把结果写回热层（copy-up）。这一机制的关键不在"能不能落盘"，而在**调度与观测**：何时落盘、落多少、失败如何处理、外部如何看到状态。

2.4.2 内存映射（mmap）

冷层读取可选择 mmap：把文件映射到进程地址空间，由操作系统页缓存承担缓存职责，

避免用户态缓冲区复制。代价是页错误不可控、内存统计不直观，因此系统把 mmap 作为可选项，

并在其上层提供块缓存以稳定命中率。

2.4.3 块缓存策略：LRU 而非 ARC

块缓存按块缓存解码后的数据。候选策略包括 LRU、LFU、ARC 与 TinyLFU。

项目早期评估过 ARC（Megiddo 与 Modha，2003）[17]——它通过 B1/B2 幽灵队列感知"被淘汰键的再次访问"，

对突发访问模式更友好；但块缓存的访问局部性强、容量以条目计，ARC 的收益不足以抵消实现与维护成本，

最终采用 LRU（访问序链表）。

表述边界：ARC 是**评估过的备选方案**，不是本系统的实现；引用它用于说明选型过程，而非声称具备自适应缓存能力。

2.4.4 异步迁移与背压

迁移在线程池中执行，客户端线程不参与磁盘 IO；当内存达到临界水位时，系统对写入施加背压，

以避免"迁移追不上写入"导致的内存溢出。背压是一种显式的降级策略，

其触发与解除都需要被观测（见 3.7.2）。

2.5 分布式与一致性技术

2.5.1 分片

分片有两个层次：进程内按 key 哈希分片（保证同 key 串行），集群级按 key 范围划分为 Region，

每个 Region 对应一个复制组。前者解决并发正确性，后者解决容量与迁移单位。

2.5.2 Raft 复制

每个复制组内由 Raft 决定日志顺序：写入先追加日志，多数派确认后提交并应用到状态机。

领导者在任期内唯一，日志按任期与索引比较，保证已提交条目不被回滚。

证据边界：本项目的 Raft 实现与测试均在**单机多进程**环境下完成，

网络分区由进程或传输层模拟，未覆盖真实跨机网络抖动与时钟漂移。

因此本文只声称"实现了 Raft 复制路径并通过混沌测试"，不声称生产级跨机共识能力。

2.5.3 时间戳与两阶段提交

跨分片事务采用"两阶段提交 + 全局时间戳"的骨架：协调者分配开始时间戳，参与者在写入时加锁并记录时间戳，

提交阶段写入提交时间戳并释放锁；崩溃或超时由后台锁清理机制依据事务状态决定提交或回滚。

这一骨架借鉴了 Percolator 的设计思想（Peng 与 Dabek，2010）[13]——用"时间戳 + 锁记录"替代阻塞式两阶段提交。

但本项目没有实现 Percolator 的多版本存储与快照读：时间戳用于事务可见性与锁判定，

读请求得到的是某一时间戳下"已提交"的视图，不提供线性一致读。

2.5.4 MVCC 的基本概念与本文边界

多版本并发控制（MVCC）通过为每个数据项保留多个版本，使读不阻塞写、写不阻塞读，

并用可见性规则决定某事务能看到哪个版本。完整实现需要版本存储、版本清理（GC）与快照管理。

本项目**只实现了时间戳与可见性判定的最小子集**，存储层保留单版本；

因此本文不使用"完整 MVCC"这一表述。

2.6 系统总体架构设计



图 2-1 Tiering-KV 系统总体架构（接入层 / 数据平面 / 控制平面）

2.6.1 三个层面

接入层：浏览器（Vue 控制台）、`redis-cli` / SDK（RESP2/RESP3）、运维脚本（HTTP JSON）。

接入层不参与存储逻辑，只负责协议转换与请求发起。

数据平面：Netty 接入、命令层（按 key 分片执行）、存储引擎（预写日志 → 内存表 → 不可变表 → 合并）、

冷热分层（水位、迁移、淘汰、块缓存），以及可选的复制与事务层。数据平面的唯一职责是"正确存取数据"。

控制平面：Spring Boot 服务 + MySQL 元数据库 + Vue 控制台，负责认证授权、实例纳管、指标采集、

键空间浏览、在线终端、告警、备份恢复与审计。控制平面不持有业务数据，只持有元数据与操作记录。

2.6.2 边界与数据流

两条数据流贯穿系统：

- **数据流**：客户端 → Netty → 命令层 → 存储引擎 →（未命中）冷层 → 回源写回热层；
- **观测流**：内核计数器 → `INFO` 文本（含按需段落）→ 控制面解析 → 指标存储 → 界面/告警/审计。

两条流在物理上分离、在契约上耦合：控制面只依赖 `INFO` 的文本格式，不依赖内核内部结构。

这一设计使"界面数字不对"可以被快速二分定位。

2.6.3 核心技术选型

表 2-2　系统核心技术选型及应用

技术	应用位置	选型依据	备注（边界）
Java 21	核心运行环境	并发原语、可观测工具、生态成熟	GC 抖动由显式内存记账缓解
Netty	网络接入	事件驱动、编解码可插拔	需自建响应保序机制
RESP2/RESP3	协议兼容	生态工具可直接连接	支持常用子集，非全集
LSM-Tree	存储主结构	写入吞吐与有序存储	合并策略为简化版
mmap	冷层读取（可选）	减少一次数据拷贝	与块缓存并用
LRU	块缓存	访问局部性强、实现简单	ARC 为评估项，未采用
Raft	多副本共识	可解释、易实现、社区验证充分	单机多进程验证
HLC	时间戳管理	物理与逻辑结合、单调可比	不等于完整 MVCC
两阶段提交	跨分片事务	语义清晰、可恢复	借鉴 Percolator 思路，未实现快照读
Spring Boot	控制平面	快速组织 REST/鉴权/定时任务	与数据面进程隔离
MySQL 8	元数据存储	事务与查询能力成熟	仅存元数据与审计，不存业务数据

2.7 本章小结

本章确定了系统的技术底座与总体结构。技术选择遵循三条原则：

1. **与负载特征匹配**：LSM 对应"写多、内存受限"的负载；LRU 对应块缓存的强局部性；
   Raft 对应"多副本、可恢复"的可靠性诉求。
2. **可解释、可验证**：选择 Netty、Raft、LSM 这些机制清晰的技术，便于在论文中展开论证；
   对每个选型都记录了备选方案与放弃理由（如 ARC、leveled 合并）。
3. **边界明确**：对"评估过的方案"与"实际实现的方案"、"单机验证"与"跨机验证"、"时间戳可见性"与"完整 MVCC"逐项标注边界，避免后文出现能力升级。
   后续章节按"总体 → 实现 → 验证"展开：第三章实现存储引擎与冷热分层，
   第四章实现控制平面与可观测性，第五章讨论分布式与一致性，第六章给出测试与实验结果。

本章引用文献

本章引用 [1][2][11][13][14][16][17]，完整条目见文末《参考文献》；协议与网络库规范见文末"技术规范"（S1、S2）。

第三章　系统核心存储引擎设计与实现

论文正文初稿 • 对应基线 6439ecf（Phase 11 之后）

图表：图 3-1（存储数据流）、图 3-2（核心存储模块类图）

数据来源：docs/benchmark/*、docs/testing/*、本仓库实测（口径见 §3.9）

3.1 设计目标与问题背景

3.1.1 为什么自己实现 LSM 主链路

KV 存储的教学实现通常止步于"内存哈希表 + RESP 协议"，其问题不在功能而在**容量模型**：

数据必须整体驻留内存，一旦超过堆上限就不可用。另一方面，直接引入 RocksDB / LevelDB 作为存储层，

虽然能获得成熟的分层能力，但核心机制成为黑盒，无法在论文中展开论证，也难以界定本人工作量。

因此本章的设计起点是：**在可控复杂度内，完整实现一条可解释、可观测、可验证的 LSM 写入与读取链路，**

再在其上叠加冷热分层，使"内存只保留热数据"成为结构性保证，而不是靠调参。

存储引擎的读写权衡是数据系统设计的基础议题 [18]。LSM-Tree 的核心思想早在 1996 年由 O'Neil 等人提出：把随机写转化为顺序写，

用多层有序结构加后台合并换取写吞吐 [1];Bigtable 进一步给出了"内存表 + 不可变 SSTable + 合并"

的工程化形态 [2]，本项目的 WAL / MemTable / SSTable / Compaction 四段式结构与之一致。

3.1.2 设计目标

编号	目标	衡量方式
S1	写路径不丢已确认的数据	强杀后 WAL 重放，逐键校验
S2	内存占用受配额约束	水位线 + 背压，2 MB 配额下 2 万次写入不超限
S3	冷数据可按需回源	冷读命中率、copy-up 行为
S4	状态可被外部观测	INFO # Tiering 全部字段来自真实计数
S5	复杂度可控	单机单进程可完整运行，组件间依赖单向

3.1.3 非目标

- 不追求与 Redis 同等的内存写性能（实测为 Redis 的 0.36–0.47×，见 §3.9）;
- 不实现完整 MVCC 快照隔离：版本号仅用于写入顺序与 TTL 守卫；
- 不做跨机分布式存储引擎（集群能力见第五章，且为单机多进程验证）。

3.2 总体方案与模块划分



图 2　存储引擎数据流：写入（WAL→MemTable→Flush→Compaction）与读取（热层→BlockCache→SSTable）

图 3-1　存储引擎数据流：写入（WAL→MemTable→Flush→Compaction）与读取（热层→BlockCache→SSTable）

3.2.1 分层结构

存储引擎自下而上分为四层（图 3-2）：

层	组件	职责
持久化层	`WALManager` / `WALWriter` / `SegmentManager`	顺序追加、分段、落盘、恢复
内存层	`MemTable` / `MemoryManager` / `SkipList`	有序内存表、字节记账、TTL
冷层	`ColdStorageEngine` / `SSTableReader` / `SSTableWriter` / `CompactionManager` / `BlockCache`	不可变表、索引与布隆过滤、合并、块缓存
编排层	`TieringController` / `WatermarkManager` / `FlushScheduler` / `MigrationScheduler` / `StorageMetrics`	水位判定、调度、指标

3.2.2 依赖方向与装饰器

所有上层能力都通过 `StorageEngine` 接口接入，装饰器模式串联：

```
StorageEngine (interface)
  └─ AbstractStorageDecorator (abstract, delegate)
       ├─ TieringStorageEngine      // 写路径：水位判定 + Flush/迁移调度
       └─ TieringReadStorageEngine  // 读路径：热层未命中则查冷层并 copy-up
```

选择装饰器而非"在 MemTable 里判断冷层"的原因是：**读写路径的关注点不同**。写入要决定"何时落盘"，读取要决定"去哪里找"，把两者拆成两个装饰器后，每个类的职责单一，也便于分别测试（`TieringStorageEngineTest` / `TieringReadStorageEngineTest`）。

3.2.3 关键选型与理由

决策点	选择	备选与理由
内存索引	16 分片跳表（无锁读 + 段写锁）	红黑树并发写锁粒度大；跳表实现简单、范围扫描天然有序
键序	`Arrays.compareUnsigned`（无符号字节序）	与 RESP 二进制键一致；避免字符串解码开销与本地化差异
冷层格式	块 + 块索引 + 布隆过滤 + CRC 的自定义格式	直接复用 SSTable 开源格式会引入外部依赖，且不利于论文解释
块缓存策略	LRU（accessOrder）	评估过 LFU 与 ARC[17]（ADR-0012），块缓存访问局部性强、ARC 的 ghost 机制收益有限，最终取 LRU（ADR-0028）
合并策略	表数阈值触发全量/分层合并	与工业界 size-tiered 思路一致，实现成本低于 leveled

3.3 WAL：顺序持久化与恢复

3.3.1 设计目标

WAL 的唯一职责是**在返回成功之前，把这次修改变成可重放的事实**。

它不参与查询，不保证"立即落盘"，但必须让"丢失窗口"可度量。

3.3.2 结构

元素	实现
段文件	`%06d.log`，默认单段 64 MB，按序号升序枚举
记录	操作类型、时间戳、键、值、TTL、单调序号；`WALRecord.encode/decode`
完整性	CRC 校验；损坏记录计入 `RecoveryStats.corruptedRecordsDiscarded` 并跳过
写缓冲	`BufferedOutputStream`（64 KB）+ `FileChannel.force(true)`
落盘策略	`ALWAYS` / `EVERY_SEC`（默认）/ `NO`

3.3.3 关键实现：空闲期也必须落盘

最初 `EVERY_SEC` 的实现是"在 append 时判断距上次 force 是否 ≥1 秒"，

这意味着**最后一次写入之后的空闲期不会触发落盘**：数据留在 64 KB 用户态缓冲里，

进程一旦被强杀就永久丢失。故障注入实测：写入 100 个键、等待 1.8 秒、强制终止，重启后**只剩 1 个键**。

修复方式是在 `WALManager` 中引入后台调度器（守护线程，500 ms 周期），并给 `WALWriter` 增加 `dirty` 标记：

清单 3-1 WALWriter 的 dirty 标记与按需落盘

```
// WALWriter：只有存在未落盘写入才真正 force，避免空闲期空转
public void force() throws IOException {
    if (!dirty) return;
    segments.current().force();  // flush + channel.force(true)
    dirty = false;
}
```

修复后同一场景 **100/100 键存活**；10 轮采样得到的后台落盘延迟为 **403 – 421 ms**（最坏 420.8 ms）。

这给出一个可写入论文的定量结论：**进程崩溃最多丢失最近一个调度窗口内的写入**，

而不是"可能丢失任意量"。

3.3.4 恢复流程

```
启动 → 枚举段文件（升序）→ 逐条解码 → CRC 校验 → 按序号重放至 MemTable
    → 应用 TTL（按记录时间戳 + ttl 计算绝对过期点）→ 返回 RecoveryStats
```

恢复不重建索引，只回放；冷层数据不在 WAL 中，由 Manifest 恢复，两者互不干扰。

3.3.5 局限

- EVERY_SEC 的语义是"最多丢一个窗口"，**不等于**"掉电安全"：force() 同时 flush+fsync，
  但本项目未做掉电注入，不对掉电场景作承诺；
- 单段 64 MB 固定，不支持按写入速率自适应；
- 恢复时间与 WAL 体积线性相关，尚无 checkpoint 加速（CheckpointManager 已实现但默认未启用）。

3.4 MemTable：内存有序表与内存记账

3.4.1 设计目标

MemTable 要在**支持并发读写**的前提下保持**有序**（供范围扫描与 Flush 顺序写出），并把内存占用变成**可查询的精确值**（水位判定依赖它）。

3.4.2 结构

- 16 个分片，每片由 SkipList 承载，按无符号字节序排序；
- 每个条目是 KeyValueEntry：键、值、时间戳、TTL、版本号；
- 内存记账由 MemoryManager 维护 AtomicLong usedBytes，写入加、删除/过期减；
- size() 返回 O(1) 计数器值，**不做遍历**（这是 INFO 能够在亚毫秒级返回的前提）。

3.4.3 内存记账与水位

MemoryManager 只回答两个问题：当前用了多少、上限多少。

水位判定交给 WatermarkManager：

表 3-1　水位状态与对应动作

状态	条件（默认）	动作
NORMAL	< 70%	无
HIGH	≥ 70%	触发 Flush
MIGRATE	≥ 85%	启动异步迁移
CRITICAL	≥ 95%	背压：限写或拒绝

把"记账"和"决策"分开，使得水位策略可以独立单测（WatermarkManagerTest 覆盖各边界），

也方便在演示实例（24 MB 配额）与默认实例（1 GB）之间切换而不改逻辑。

3.4.4 TTL 的处理

TTL 不靠后台扫描删除，而是**读取时惰性判定**：MemTable 在 get 时比较 now 与条

目过期点，

过期即视为不存在；TTLManager 负责定期回收以释放内存。

这样避免了"定时任务扫描全表"的抖动，代价是过期数据会短暂占内存。

3.4.5 局限

- 分片数固定 16，不能随核数扩展；
- 版本号只用于写入顺序，未实现快照读；
- 大 value 直接驻留内存，没有大对象下沉策略。

3.5 SSTable：不可变磁盘表

3.5.1 设计目标

SSTable 一旦写出就不再修改，因此它的设计目标是：**顺序写、范围读、少读块**。

3.5.2 文件布局

清单 3-2 SSTable 文件布局

```
+----------------+  数据块（blockTargetBytes，默认 4 KB）
|   Data Blocks  |  块内按字节序排列的 KeyValueEntry
+----------------+
|   Block Index  |  每块首键 → (offset, length)
+----------------+
|   Bloom Filter |  m 位数组 + k 个哈希，误判率由 bitsPerKey 控制
+----------------+
|     Footer     |  索引/Bloom 的偏移与长度、魔数
+----------------+
```

块级 CRC 用于检测损坏；布隆过滤器借鉴 Bloom 的经典结论 [16]：

用少量位数组即可把"键一定不存在"的判断前置，避免无效块读。Bigtable 也在 SSTable 中采用了同样的组合 [2]。

3.5.3 写与读

写：SSTableWriter 顺序追加条目，块满即落盘并记录索引；finish() 写索引、Bloom、Footer，

返回 SSTableMeta（id、条目数、字节数、首末键）。

读：SSTableReader 先查块索引定位块，再在块内二分查找（Keys.compare），最后校验 CRC。

冷层读取可走 mmap（useMmap），把页缓存交给操作系统；上层 BlockCache 则缓存解码后的块。

3.5.4 元数据与键范围

SSTableMeta 是 record，字段为 id / fileName / entryCount / fileSize / firstKey / lastKey。

其中 firstKey/lastKey 是**写入顺序的首末条目**，只有在"写入有序"的前提下才是范围边界——

这一点由 Flush 与 Compaction 保证（两者都按字节序遍历）。

工程细节：INFO sstables 输出键范围时使用 Base64，并在协议中显式声明 sstable_key_order:bytewise。

原因是数字型键（tier:1000000 与 tier:999999）在字节序下呈现"看起来反了"的顺序，若不声明比较器语义，读者会误判为缺陷。

3.5.5 局限

- 无块级压缩，磁盘占用高于工业实现；
- 单表最大条目数受 Flush 触发时机影响，没有按大小切分；
- 读放大为 O(表数)，未做分区索引（bloom 只能减少无效表访问）。

3.6 Compaction：合并与空间回收

3.6.1 设计目标

合并要解决两个问题：**表数量增长导致读放大**，以及**旧版本与墓碑占用空间**。

3.6.2 触发与执行

- 触发条件：`tables.size() >= compactionThreshold`（默认 8）；
- 全量合并 `compactAll()` 与分层合并 `compactLeveled()` 两条路径；
- `CompactionTask` 对输入表做多路归并，按字节序输出，保留最新版本；
- 完成后 `installCompaction` 删除输入文件、重写 Manifest、替换表清单；
- 通过 `CompactionListener` 回调 `StorageMetrics`，使指标与行为同步。

3.6.3 并发安全

表清单的读取与替换共用同一把锁：`tablesSnapshot()` 返回 `List.copyOf(tables)` 的不可变副本，

`installCompaction` 在锁内整体替换。因此外部观察到的永远是"合并前"或"合并后"的完整状态，

不会引用已删除的表。

该性质由测试固定：写线程持续 `writeTable` + `compactAll`，3 个读线程循环取快照并校验

"逐行之和 == 总计"，1500 ms 内无一次撕裂（Phase 9）。

3.6.4 实测

场景	输入	输出	耗时
演示实例一次全量合并	429.9 MB	191.6 MB	1,487 ms

输出显著小于输入，说明覆盖版本与墓碑被回收；该组数字直接来自 `INFO # Tiering` 的

`tiering_compaction_input_bytes / output_bytes / latency_ms`，不是事后统计。

3.6.5 局限

- 阈值为固定表数，不感知单表大小，可能出现"少量大表"或"多量小表"；
- 合并期间不限制写入速率，极端情况下会与 Flush 争抢磁盘；
- 未实现增量（leveled）合并的完整策略。

3.7 冷热分层：水位、迁移与缓存

3.7.1 设计目标

让"内存占用"成为**可配置的硬约束**：无论写入多少数据，内存使用都不超过配额，超出部分自动下沉到冷层，并在读取时按需回源。

3.7.2 触发链路

```
写入 → MemTable（记账）→ WatermarkManager.evaluate
    ├─ HIGH      → FlushScheduler.scheduleFlush() → SSTable
```

```
├─ MIGRATE   → MigrationScheduler.submit() → 迁移到冷层
└─ CRITICAL  → BackPressureController → 限写/拒绝
```

三个动作都是**后台执行**，客户端线程从不做磁盘 IO——这一点由 Phase 6 实验验证：

2 MB 配额下连续写入 2 万次，客户线程未出现阻塞，峰值内存 350 KB。

3.7.3 读取回源

`TieringReadStorageEngine` 拦截读请求：

1. 先查 MemTable；
2. 未命中则通过 `UnionStorageIterator` 合并冷层表（新→旧）；
3. 命中后把条目 `copy-up` 回热层，使后续访问命中内存。

`UnionStorageIterator` 的存在使得"多表覆盖同一键"的场景仍返回最新值，这也是 `INFO sstables` 中多张表键范围重叠却语义正确的原因。

3.7.4 BlockCache

冷层块缓存采用 LRU（`accessOrder=true`），按块缓存解码后的 `ByteBuffer`，容量以条目数计。指标包括 hits / misses / evictions / hitRate，全部进入 `INFO`。实测两个极端：

场景	数据集	缓存	命中率
演示实例	597 MB 冷数据	默认容量	84.8%
压力场景	232 MB 冷数据	小容量	3.2%

两者都不是"错误"，而是"缓存容量 / 工作集"的真实比例；界面如实展示，不做平滑或隐藏。

3.7.5 局限

- 迁移按热点排序，但热点统计窗口固定，突发流量下可能迁移"刚热起来的键"；
- 淘汰与迁移共用配额，临界水位下的策略是"先限写再淘汰"，未做更细的 QoS；
- BlockCache 未做字节级容量控制，与内存配额存在口径差异。

3.8 核心模块类图（图 3-2）

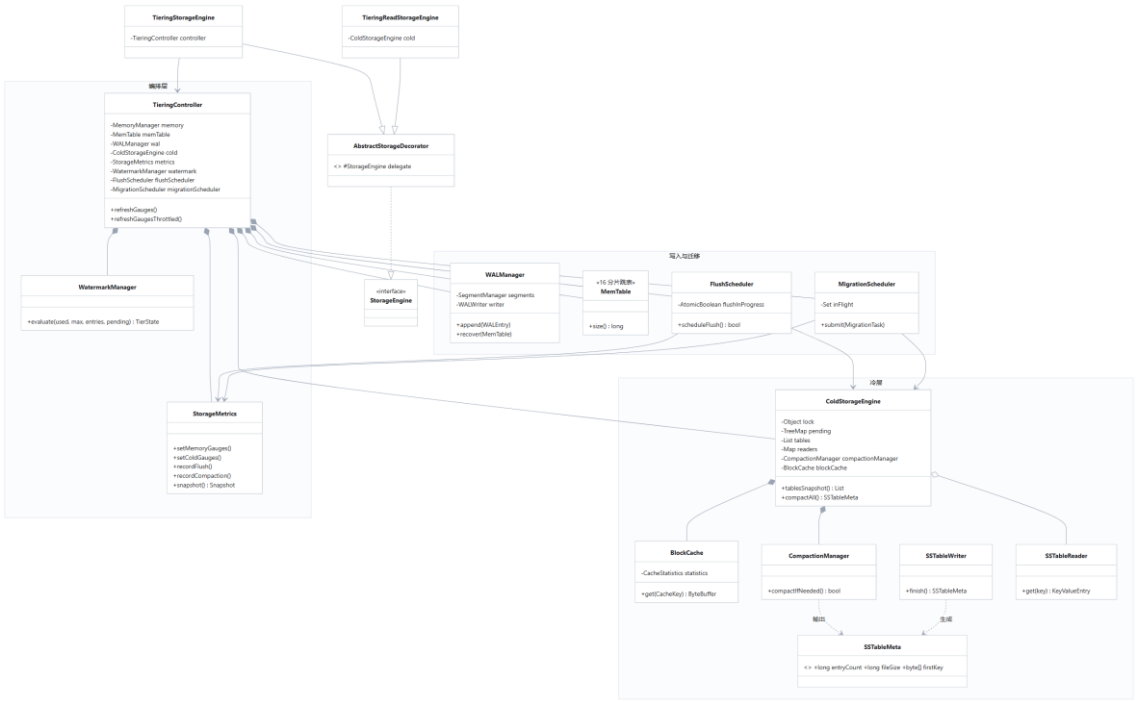


图 3-2　核心存储模块类图（编排层 / 写入与迁移 / 冷层；关系取自实际字段与继承）

类图说明三点：

1. **继承关系只有一处**：TieringStorageEngine 与 TieringReadStorageEngine 都继承 AbstractStorageDecorator，后者实现 StorageEngine 接口；其余均为组合或依赖；
2. **组合关系体现生命周期**：TieringController 持有 MemTable/WALManager/ColdStorageEngine/StorageMetrics/两个调度器，随控制器创建与关闭；ColdStorageEngine 持有 BlockCache 与 CompactionManager；
3. **依赖关系体现调用**：FlushScheduler 与 MigrationScheduler 都依赖 ColdStorageEngine 与 StorageMetrics，但彼此不知道对方存在，调度解耦。

3.9 实验与结果

3.9.1 环境

项	值
硬件	Windows 11 • 20 逻辑核 • 31.6 GB 内存 • SSD（本地基准）；Docker bridge 网络（对比基准）
JDK	21（Microsoft OpenJDK 21.0.12）
引擎配置	16 分片；WAL 段 64 MB、fsync=EVERY_SEC；compaction 阈值 8；水位 70/85/95
堆	默认 -Xmx1g；分层专项实验 -Xmx2g（Redis 对比时双方一致）

3.9.2 与 Redis 7 的横向对比

表 3-2　与 Redis 7 的吞吐与延迟对比（同机、同客户端）

场景	Tiering-KV	Redis 7	比值

SET • c=1	10,347 rps / p50 0.079 ms	21,834 / 0.039 ms	0.47×
GET • c=1	11,148 / 0.079 ms	23,641 / 0.039 ms	0.47×
SET • c=50	59,701 / 0.431 ms	165,975 / 0.159 ms	0.36×
GET • c=50	63,512 / 0.423 ms	161,160 / 0.159 ms	0.39×

口径：同机、同客户端（`redis-benchmark`）、同 Docker 网络；`-n 200000 -c {1,50} -r 1000000 -d 64`；

Redis 关闭 RDB/AOF，Tiering-KV 保持默认 WAL——即本项目承担持久化成本而 Redis 不承担。

因此该比值应理解为"在承担 WAL 的前提下，达到 Redis 的 1/3 左右"，而非"性能等价"。

3.9.3 分层调度

表 3-3 Flush 与异步迁移的吞吐

场景	规模	吞吐	备注
自动 Flush	100K 条	857K entries/s	116.6 ms
自动 Flush	200K 条	844K entries/s	237.0 ms
异步迁移	100K 条	297K ops/s	成功率 100%，端到端 P99 69.5 ms
异步迁移	1M 条	308K ops/s	成功率 100%，端到端 P99 1,397.9 ms

P99 为"提交→完成"的端到端延迟，包含提交快于消费时的队列堆积；单任务执行延迟远低于此。

3.9.4 持久化边界

表 3-4 WAL 持久化边界的实测结果

实验	结果
写入 100 键 → 等 1.8 s → 强制终止 → 重启	修复前 99/100 丢失；修复后 100/100 存活
后台落盘延迟（10 轮采样）	403 – 421 ms（最坏 420.8 ms）
空闲 2 s 文件大小	不变（`dirty` 标记避免空转 fsync）

3.9.5 存储结构与清单成本

表 3-5 不同表数量下的清单成本

表数	快照（`List.copyOf`）	渲染（文本+Base64）	输出
1	7 μs	1,949 μs（含 JIT）	191 B
10	7 μs	96 μs	953 B
100	9 μs	573 μs	8,797 B
1,000	52 μs	2,236 μs	97,003 B

3.9.6 小结

本引擎在**承担 WAL 持久化**的前提下达到 Redis 约 1/3 的吞吐，

分层调度在 100K – 1M 规模下保持 100% 迁移成功率，内存严格受配额约束，

崩溃恢复的丢失窗口被量化为"一个 500 ms 调度窗口"（实测 403–421 ms）。这些结论共同支撑了设计目标 S1–S5，但 S1 仅覆盖**进程崩溃**，不含掉电。

3.10 本章局限

1. 掉电恢复未验证；
2. 合并策略为固定表数阈值，非 size-tiered；
3. 无压缩，磁盘占用偏高；
4. BlockCache 为 LRU，未采用 ARC（评估见 ADR-0012）；
5. 冷层 `DBSIZE` 需要全量迭代（实测 17.1 ms 首次），未做增量计数。

本章引用文献

本章引用 [1][2][16][17][18]，完整条目见文末《参考文献》。

第四章　系统控制与管理模块设计与实现

论文正文初稿 • 对应基线 6439ecf

图表：图 2-1（总体架构）、图 4-1（INFO 时序）、图 4-2（备份恢复时序）

数据来源：docs/testing/phase10-e2e.md、docs/api/openapi.json、本仓库实测

4.1 设计目标与问题背景

4.1.1 为什么需要独立的控制面

内核是一个 RESP 服务，只认命令、不认人：它能回答"这个键是什么"，但回答不了

"谁在什么时候改了它""现在有几个实例、哪个掉线了""上次备份能不能恢复"。

如果把这些能力塞进内核进程，会带来两个问题：

1. **故障域扩大**：控制台的 bug（如一次慢查询、一次 OOM）会拖垮数据面；
2. **协议污染**：管理类命令会进入 Redis 命令空间，破坏兼容性。

因此本项目把控制面做成**独立进程**（Spring Boot），只通过 RESP 与内核通信（图 2-1）。

这条边界带来两个直接收益：内核可单独升级/重启；控制面可独立部署与限流。

4.1.2 设计目标

编号	目标	衡量方式
C1	协议兼容、零侵入	只用标准 RESP 命令，不新增管理命令（INFO <section> 除外）
C2	观测数据可信	所有指标来自内核计数，缺失时显式提示而非补零
C3	操作可追溯	登录/写入/实例/告警/备份/终端全部落审计
C4	失败状态明确	401/403/404/409/502/503 各有明确语义
C5	多实例隔离	切换实例后指标、图表、会话、历史全部跟随

4.1.3 非目标

- 不做多租户（所有登录用户看到同一批实例）；
- 不做告警的通知渠道（邮件/短信），仅落事件表；
- 不做跨集群联邦管理。

4.2 RESP 协议与 Netty 接入

4.2.1 协议实现

内核实现了 RESP2 与 RESP3 的常用子集：简单字符串、错误、整数、批量字符串、数组、

以及 RESP3 的 map/set/push 类型。协议解析由 RespRequestParser 完成，

命令分发由 CommandEngine 负责。

选择自己实现而非引入 Lettuce/Jedis 服务端库的原因是：**我们要控制执行语义**

——

分片调度、事务排队、慢命令计量都在这一层，借用现成框架反而不好插入。

4.2.2 Netty 接入与并发模型

清单 4-1 Netty 接入管线

```
ChannelPipeline: 解码器 → CommandHandler → CommandEngine → KeyShardExecutor(16)
                                                  ↓
                                  ResponseSequencer（保序）→ writeAndFlush
```

- CommandHandler 继承 ChannelInboundHandlerAdapter，在 channelRead 中解析并执行；
- CommandEngine 把命令按 key 哈希到 16 个执行器之一，保证**同 key 串行**；
- ResponseSequencer 解决"多分片并行执行、单连接按序返回"的问题：
  每条命令带序号，只有队首完成才写出，避免响应乱序。
  并发模型的选择依据是"**单连接语义正确优先于极限吞吐**"：
  Redis 客户端默认假设响应与请求一一对应且有序，乱序会让上层 SDK 直接错乱。

4.2.3 命令层与扩展

157 个命令名由 CommandCatalog 注册，按族组织（string / hash / list / set / zset / stream /
geo / bit / json / ts / vector / 管理类）。每个命令实现 Command 接口：

清单 4-2 命令接口

```
RespValue execute(List<byte[]> args, StorageEngine storage);
```

多模型命令（json、ts.*、vector.*）通过 MultiModelCommand 统一转发，
避免为每个子命令写一个类。

4.2.4 局限

- 未实现 RESP3 的全部类型与 pub/sub 推送语义；
- 分片数固定 16；
- 慢命令只计量不熔断。

4.3 命令执行与存储引擎的边界

4.3.1 接口收敛

所有命令只依赖 StorageEngine 接口，不直接触碰 MemTable 或 SSTable。
因此存储层可以替换（如加 VectorIndex 装饰器）而不影响命令。

4.3.2 事务与集群的接入点

事务（MULTI/EXEC/WATCH）在命令层拦截并排队；集群模式由 ReplicatedStorageEngine 包装存储接口，
把写请求转成 Raft 日志。两者都不修改命令实现本身。

4.3.3 局限

跨节点事务使用两阶段提交 + 全局时间戳 ^{[19][20]}，借鉴 Percolator 的设计思想 [13]，
但**未实现其多版本存储与快照读**（详见 5.5.3 与 5.6.4）。
一致性验证限于单机多进程。

4.4 INFO 可观测性：从内核计数到界面数字

4.4.1 设计目标

让"界面上看到的每个数字"都能追溯到内核的一个计数器或表元数据，并且**缺失、为零、失败**三种情况必须可区分。

4.4.2 数据来源分层

类别	例子	语义	重启后
Gauge	MemTable 用量、SSTable 数量、磁盘占用	瞬时值	由 WAL 重放 + Manifest 重建
Counter	Flush 次数、Compaction 次数、迁移成功/失败、缓存命中	自启动累计	归零
直方图	P50/P95/P99/max	17 桶 + 60 s 滚动窗口	归零

4.4.3 请求时序

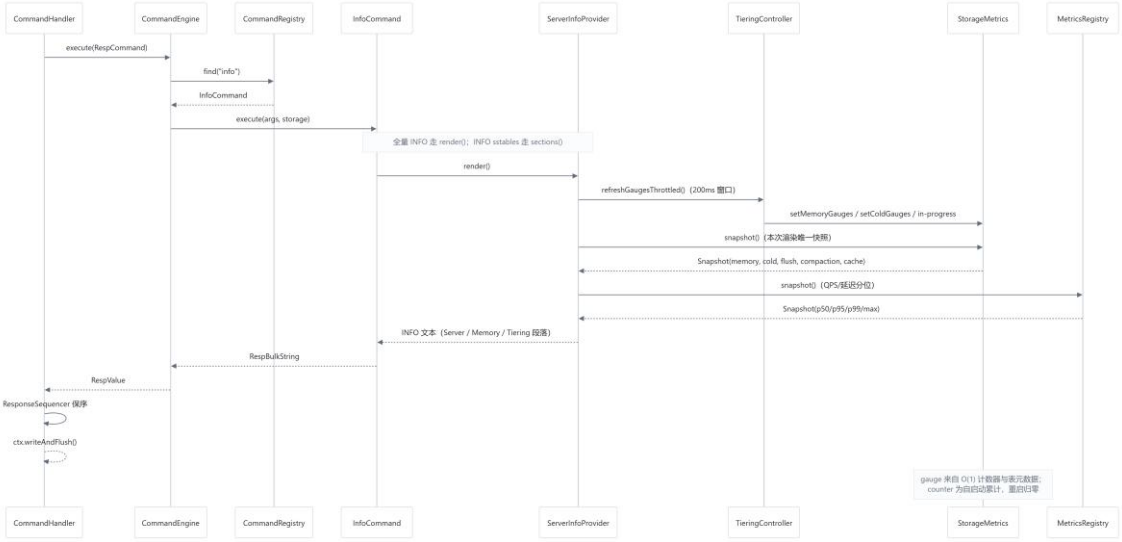


图 4-1 INFO 请求处理时序（含 gauge 刷新、单快照、延迟分位读取）

关键设计点有三：

1. **节流刷新**：`refreshGaugesThrottled()` 在 200 ms 内复用上次结果，避免高频 INFO 反复复制表清单；
2. **单快照**：`render()` 开头刷新一次 gauge 并取一次 `StorageMetrics.Snapshot`，`# Memory` 与 `# Tiering` 共用，杜绝"内存 3.1 MB / MemTable 3.4 MB"这类自相矛盾；
3. **锁外编码**：快照是 `List.copyOf` 的不可变副本，Base64 与字符串拼接都在锁外完成。

4.4.4 延迟分位

延迟用固定分桶直方图而非平均值：17 个上界（50 μs – 5 s），每桶一个 `LongAdder`；`LatencyStats` 维护 current/previous 两个窗口，**读取时惰性轮转**（60 s），不需要额外线程。

分位返回桶上界，并以实测最大值封顶，避免出现 `P95 > max` 的荒谬读数；窗口

无样本时返回 0 而不是历史累计。

4.4.5 实测开销

场景	吞吐	p99
无 INFO 流量	11,452 ops/s	0.537 ms
叠加 50 次/秒 INFO	12,061 ops/s	0.503 ms

`INFO` 自身 p50 0.246 ms、p95 0.509 ms（200 次）。结论：观测对业务延迟无可测影响。

4.4.6 局限

- 分位是分桶估算，不是精确分位；
- 200 ms 节流意味着 gauge 最多滞后一个窗口；
- 冷层精确计数（`DBSIZE` 口径）在变更后首次需要全量迭代。

4.5 控制面：认证、审计与多实例

4.5.1 认证与授权

- JWT（HS256）+ BCrypt 口令；token TTL 默认 720 分钟；
- 三级角色：ADMIN / OPS / VIEWER；写接口用 `@PreAuthorize` 注解控制；
- 权限矩阵由测试固定：VIEWER 写键空间 403、读审计 403、读备份 200；匿名 401。

4.5.2 审计

审计表记录：时间、用户、动作、对象、详情、IP、成功与否。

关键实现细节是：**失败操作必须留痕**。

早期 `AuditService.record()` 与业务方法共享事务，业务抛异常时记录被一起回滚，

导致"被拦截的终端命令查不到审计"。修复方式是把该方法标注为 `REQUIRES_NEW`，使其在独立事务中提交，与调用方成败无关。

导出为 CSV，带 UTF-8 BOM 与中文表头，便于 Excel 直接打开；测试用 RFC4180 解析器校验，

而不是按逗号切分。

4.5.3 多实例隔离

控制面按实例维护独立的 RESP 连接池与指标序列；前端切换实例时，

指标、图表、终端会话标题、命令历史全部跟随。

验证方法是"双向比对"：直连内核读 INFO，再调控制面 API，逐字段比较；

两个实例各 9 项检查全部一致，且两实例数值互不相同（18/18）。

4.5.4 失败语义

表 4-1　控制面的失败状态与错误码

状态	场景	错误码
401	未认证 / token 过期或签名非法	40100
403	角色不足	40300
404	实例/规则/备份不存在	40400
409	同实例已有备份在跑	40900
502	内核不可达（连接失败）	50201

502	内核返回不可解析内容	50200
503	数据库不可用	50300

503 的引入是复核阶段的整改：数据库断连原本返回 500 + 内部术语，现在只对"事务创建失败 / 连接获取失败 / 资源不可用"三类异常映射为 503，文案为"数据库暂不可用，请稍后重试"，不暴露 SQL、驱动或堆栈。

4.5.5 局限

- 单租户，所有登录用户共享实例视图；
- 审计只覆盖写操作与登录，高频只读不落库；
- token 无刷新机制，过期后需重新登录。

4.6 备份与恢复

4.6.1 设计目标

在不阻塞内核的前提下导出一致性可接受的逻辑快照，并支持**可重复、可审计**的恢复。

4.6.2 时序

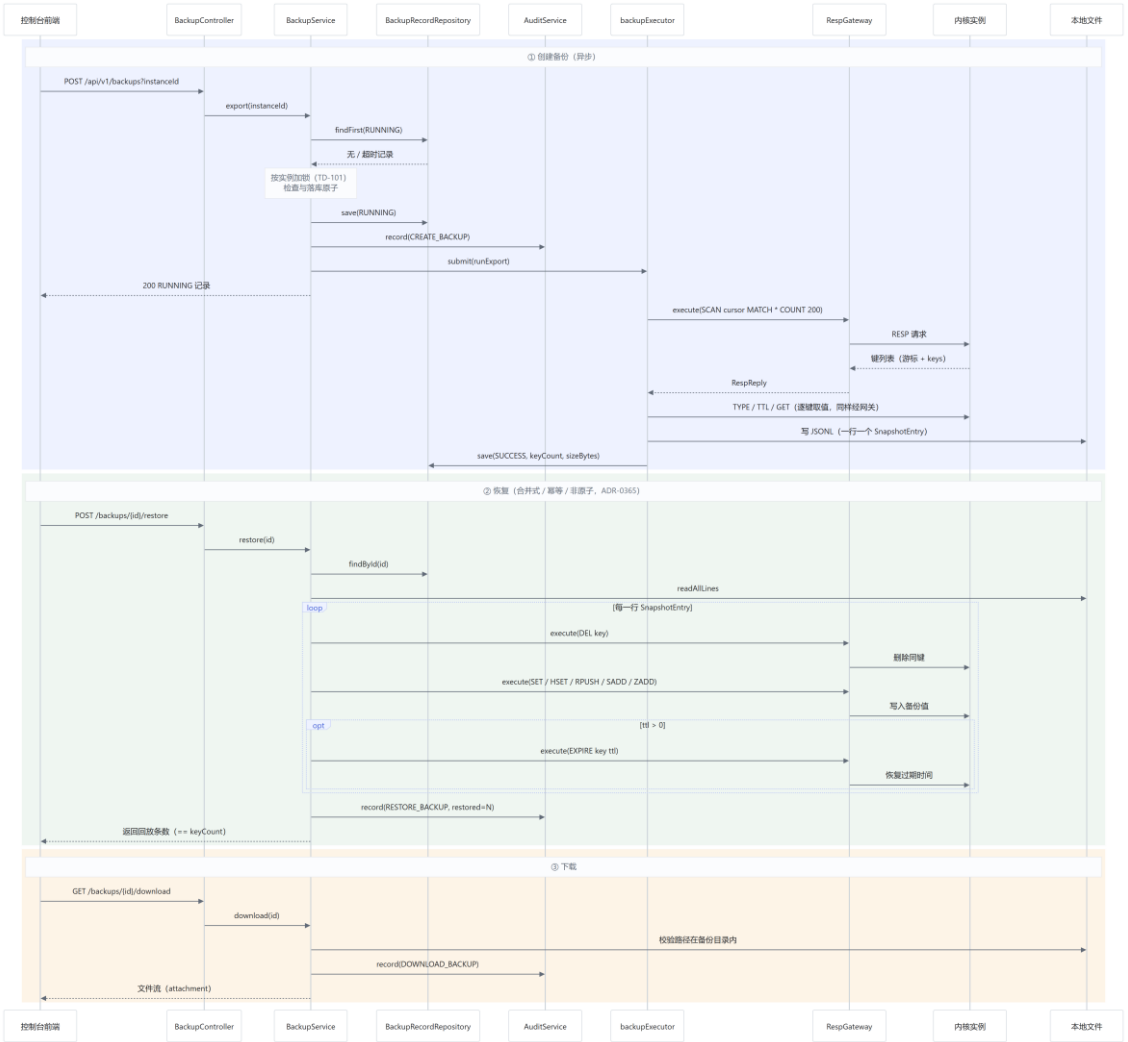


图 4-2　备份 / 恢复 / 下载时序（含鉴权、审计、网关与文件读写）

4.6.3 三个关键决策

① 异步 + 单实例串行

export() 立即返回 RUNNING 记录，真正导出在线程池中执行；

同一实例的"检查 RUNNING + 落库"必须原子，否则两个并发请求会双双成功（实测 5/5 轮复现）。

修复后并发请求稳定返回 200 + 409。

② 合并式、幂等、非原子（ADR-0365）

- 合并式：逐条 DEL 后写回，备份中不存在的键**保留**；
- 幂等：同一份备份重复恢复结果一致；
- 非原子：中途失败不回滚，已写入部分保留，并记录 result=failed restored=N。

③ 三类操作全部审计

CREATE_BACKUP / RESTORE_BACKUP（含失败）/ DOWNLOAD_BACKUP。

这是复核阶段补上的缺口：在此之前，系统能恢复数据，却无法追溯"谁创建的、谁恢复的、谁下载的"。

4.6.4 实验

项	结果
导出规模	3,123 键 / 约 400 KB JSONL
恢复完整性	回放数 == 记录 keyCount（3,123/3,123）；批量首/中/尾抽样 3/3
特殊字符	含逗号、引号的值原样还原
并发提交	修复前 200/200(5/5 轮)；修复后 200/409(5/5 轮)
合并语义	备份后新增的键在恢复后仍存在

4.6.5 局限

- 恢复无并发保护（两次恢复会互相覆盖）；
- 全量读入内存（readAllLines），上限由 MAX_KEYS 与堆大小约束；
- 无增量备份与 PITR。

4.7 本章实验与小结

4.7.1 端到端验收

表 4-2 七条业务闭环的断言与代表结果

闭环	断言数	代表结果
认证与权限	10	改密后旧口令 401；VIEWER 越权 403
实例管理	9	探测 UP/DOWN；删除后列表不含
键空间	14	TTL 到期后内核 EXISTS=0
在线终端	11	危险命令 400；历史按实例隔离
备份恢复	16	回放数 ==keyCount ； 并发 200+409
告警	10	确定性触发 → FIRING → RESOLVED

审计	21	时间/动作过滤；CSV 每行列数一致

合计 91 条断言，全部在真实内核 + 真实 MySQL 上执行。

4.7.2 故障注入

表 4-3　四类故障注入的实测结果

故障	结果
实例离线	502/50201，前端显示 – 与红色提示
内核崩溃	100/100 键存活，进程级计数归零
数据库断连	5,046 ms 返回 503+50300；恢复后 108 ms 自愈（不重启控制面）
Token	篡改→401；有效签名+过期→401；有效未过期→200

4.7.3 小结

控制面通过"只走 RESP"的边界保持了与内核的低耦合，
用"单快照 + 三态展示"保证了观测数据的可信性，
用"审计 + 明确的失败码"保证了操作可追溯，
用"异步 + 串行化 + 合并式恢复"给出了备份能力的可用语义。

4.8 本章局限

1. 单租户、无细粒度实例授权；
2. 恢复无并发保护、非原子；
3. 审计不覆盖只读操作；
4. 覆盖率 62.57%，异常分支覆盖不足；
5. 告警只有事件记录，无通知渠道。

本章引用文献

本章引用 [11][13][19][20]，完整条目见文末《参考文献》。
协议、网络库与指标语义规范见文末"技术规范"（S1、S2）。

第五章　分布式架构与一致性机制设计

论文正文初稿 • 对应基线 6439ecf

图表：图 5-1（集群与分片）、图 5-2（Raft 日志复制）、图 5-3（分布式事务协调）

依据：cluster/** 、cluster/raft/** 、transaction/** 、ADR-0037 ～ 0091 、ADR-0298/0339/0352

5.0 证据分级说明（本章的阅读前提）

分布式能力最容易"过度声明"。本章对每一项能力标注证据等级，且严格区分三档：

等级	含义	本章标注为
E1 已实现且验证	有对应实现，且有通过的自动化测试（本次全量回归覆盖）	"已实现并通过测试"
E2 已实现但验证有限	有实现与测试，但只覆盖单机多进程 / 特定传输，未做真实跨机故障	"已实现，在单机多进程环境下验证"
E3 设计/部分实现，未完成验证	有代码或 ADR，但缺少端到端验证	"设计方案，尚未完成验证"

本章不出现"生产级""跨机房容灾"这类未经本环境验证的表述。

需要强调的是：mvn test 全量回归 **15,014 次执行、0 失败**，其中包含 62 个集群测试类、

33 个事务测试类、12 个 Region 测试类；它们的通过只证明"在本机多进程模型下行为正确"。

5.1 设计目标与总体架构

5.1.1 目标

单机引擎解决了"内存装不下"，但没解决"一台机器装不下"。分布式层的目标是：

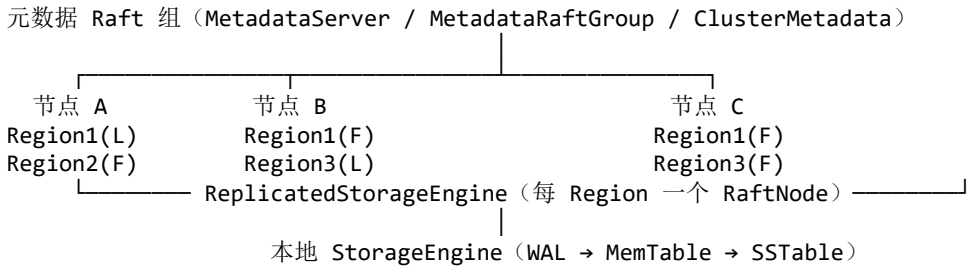
表 5-1　分布式设计目标与证据等级

编号	目标	证据等级
D1	多副本写入经共识后应用，单节点故障不丢已提交数据	E2（单机多进程）
D2	数据按 Region 分片，支持拆分/合并/迁移	E1（RegionSplitTest/RegionMergeTest）
D3	元数据自身高可用（元数据 Raft 组）	E1（MetadataTest/MetadataLeaderFailoverTest）
D4	跨分片事务具备原子性	E2（CrossNodeTransactionTest/TxnNetworkFailureTest）
D5	时间戳单调，读能看到已提交写	E1（HLC + TSO 单测）
D6	跨机房/跨云事务	E3（ADR-0109/0214/0339，仅单机模拟）

5.1.2 分层

客户端（ClusterClient）──按 Region 路由──┐

▼



关键设计：**Raft 是"存储适配器"，不是新的存储引擎**。ReplicatedStorageEngine 包装本地引擎，

写请求先追加到 Raft 日志，提交后由状态机回调 apply 到本地引擎；读请求直接读本地（ADR-0037）。

因此第三章的 WAL / MemTable / SSTable 完全不用改。

5.1.3 非目标

- 不做跨数据中心的多活写入（多地域方案见 ADR-0109/0123，属 E3）；
- 不做动态成员变更的在线扩缩容（有 Region 迁移，无节点级 rebalance 验证）；
- 不提供线性一致读（readTS 为 HLC 快照读，见 §5.5）。

5.2 集群管理与数据分片

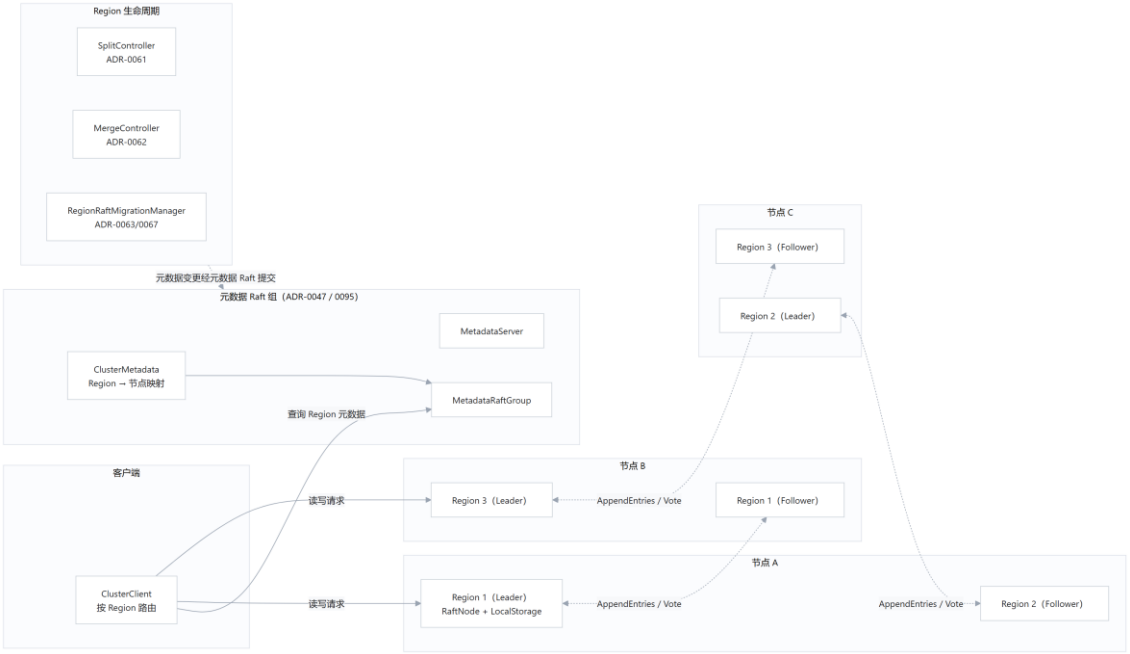


图 5-1　集群与分片架构（元数据 Raft 组、Region 与复制关系、生命周期控制器）

5.2.1 两级分片

层级	机制	说明
进程内	16 个 key 分片 + KeyShardExecutor	保证同 key 串行（ADR-0023）
集群级	Region（按 key 范围切分）	每 Region 一个 Raft 组（ADR-0057/0058）

两级分片解决不同问题：进程内分片解决**并发正确性**，Region 分片解决**容量与迁移单位**。

5.2.2 元数据

`ClusterMetadata` 维护 Region → 节点映射，由 `MetadataRaftGroup` 复制（ADR-0047/0095），

`MetadataServer` 对外提供查询与变更。客户端先查元数据，再直连 Region Leader。元数据高可用由 `MetadataLeaderFailoverTest` 覆盖（E1）。

5.2.3 Region 生命周期

操作	实现	证据
拆分	`SplitController` + `SplitSnapshot` + `SplitWriteBuffer`（ADR-0061）	`RegionSplitTest`（E1）
合并	`MergeController` + `MergeTask`（ADR-0062）	`RegionMergeTest`（E1）
迁移	`RegionRaftMigrationManager`（ADR-0063/0067）	`RegionChaosTest`（E2）
动态再平衡	负载驱动调度（ADR-0126/0132）	E3（有实现，未做多节点压力验证）

拆分期间用写缓冲承接增量写入，快照完成后回放，避免"拆分瞬间丢写"。

5.3 Raft 共识与日志复制

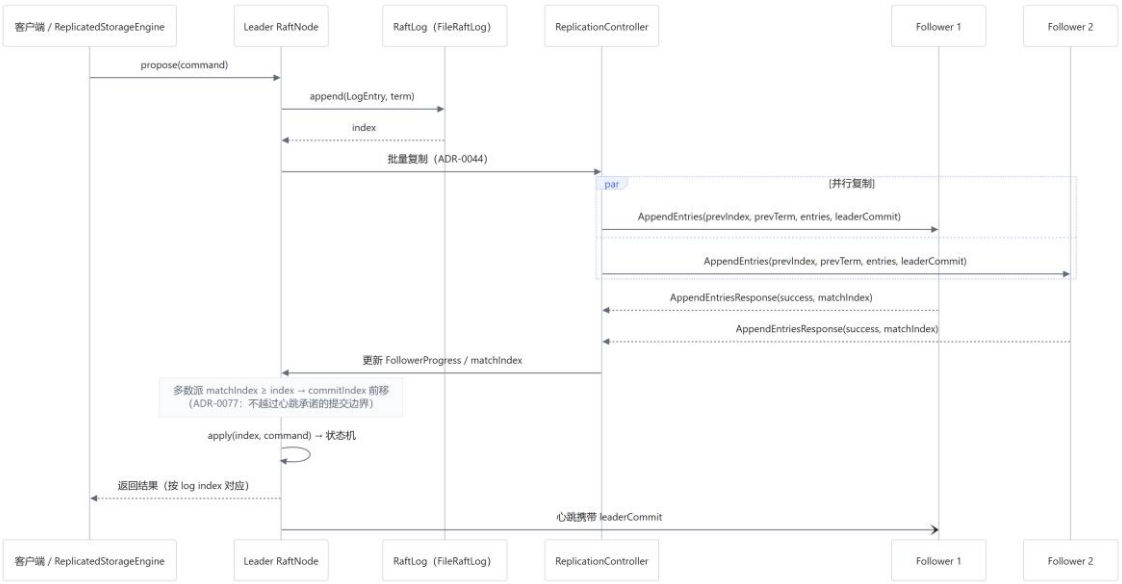


图 5-2 Raft 日志复制时序（批量 AppendEntries、多数派提交、状态机应用）

5.3.1 实现范围

`RaftNode` 是一个**完整但精简**的实现（ADR-0037～0044）[11]：

能力	实现要点	证据
角色与任期	`RaftState`、`Term`，任期单调	`RaftTest`（E1）
选举	随机化超时 + 投票请求（`VoteRequest/Response`）	`RaftTest`（E1）
心跳	固定间隔，携带 `leaderCommit`（ADR-0077）	`RaftTest`（E1）
日志复制	批量 `AppendEntries` + inflight 上限 + group	`RaftTest/ReplicatedAtomicOpsTest`（E1）

	commit（ADR-0044）	
持久化	FileRaftLog（段文件 + 编码，ADR-0039）、RaftPersistentState	Phase22DiskChaosTest（E2）
快照	SnapshotManager + InstallSnapshot（ADR-0040）	RaftTest（E1）
传输	可替换：本地内存 / Netty TCP	TcpClusterIntegrationTest（E2）
原子读改写	ATOMIC 命令入日志、按 log index 回传（ADR-0352/TD-081）	ReplicatedAtomicOpsTest（E1）

5.3.2 提交规则

- Leader 维护每个 Follower 的 matchIndex；当多数派的 matchIndex ≥ index 时推进 commitIndex；
- commitIndex 不越过"心跳承诺的边界"（ADR-0077），避免新 Leader 回滚已应用条目；
- 应用（apply）与提交分离：CommitNotifier 通知状态机按 index 顺序应用。

5.3.3 与存储层的接口

清单 5-1 ReplicatedStorageEngine 的写路径示意

```
// ReplicatedStorageEngine: 写经日志，读走本地（简化示意）
public void put(KeyValueEntry entry) {
    long index = raftNode.propose(encode(entry));   // 追加日志并等待提交
    // 提交流程中由 stateMachine 回调 apply → delegate.put(entry)
}
```

边界：Raft 只负责"日志顺序"，不负责"值语义"；覆盖、TTL、版本由本地引擎决定。

5.3.4 局限

- 未实现 Pre-Vote 与 CheckQuorum；
- 成员变更未实现联合共识（joint consensus）；
- 传输层虽有 Netty 实现，但未在真实多机网络下做分区注入（E2）。

5.4 故障检测、领导者选举与恢复

5.4.1 检测

- Follower 在选举超时内未收到心跳 → 转为 Candidate 发起投票；
- Leader 通过 FollowerProgress 判断落后程度，必要时降级；
- LeaderTransferManager 支持主动转移（TimeoutNowRequest）。

5.4.2 选举与脑裂防护

随机化超时 + 任期比较保证同一任期最多一个 Leader；
LeaderElection 只在获得多数派投票后提升为 Leader。

5.4.3 恢复路径

场景	恢复方式	证据
Follower 重启	从 FileRaftLog 重放未提交条目，向 Leader 对齐	Phase22DiskChaosTest（E2）
Leader 失效	重新选举；新 Leader 不回滚	FailoverTest/ChaosClusterTest

	已提交条目	（E2）
日志落后过多	`InstallSnapshot` 全量同步	`RaftTest`（E1）
网络分区	少数派不提交，恢复后追平	`ChaosValidationTest`（E2）

5.4.4 局限

- 全部在**单机多进程**下验证：分区用进程/传输层模拟，未做真实网络抖动与时钟漂移；
- 未验证"跨机房 RTT 下的选举风暴"这类长尾场景（E3）。

5.5 MVCC 与混合逻辑时钟（HLC）

5.5.1 为什么需要 HLC

分布式事务需要"全局可比的时间戳",但物理时钟会回拨。HLC[14] 用 `physicalMillis × 10^6 + logical`

构造单调时间戳（ADR-0072）：物理时间前进时用物理值，回拨时用逻辑计数保证**不倒退**。

5.5.2 实现

组件	职责
`HybridLogicalClock`	`now()` 返回单调时间戳；观测到更大值时追赶
`HybridClockCalibrator`	与 TSO 对齐，避免长时间漂移
`TimestampOracle` / `ResolvedTimestampService`	分配 `startTS/commitTS` 与安全读水位
`AutoTransactionExecutor`	读请求 `readTS = max(HLC.now(), oracle.peek())`

读路径的关键约束写在代码注释里：**HLC 可能略落后于 oracle 已分配的 commitTS**，

因此读水位取两者较大值，保证"已提交可见"。

5.5.3 MVCC 的实际范围

能力	状态
写入版本号（供 WAL 排序与 TTL 守卫）	E1，第三章已述
事务快照读（readTS 下可见性）	E2：`AutoTransactionExecutor` 读路径
多版本存储与 GC	E3：未实现完整 MVCC 表，仅锁记录携带时间戳

必须说明：本项目**没有**实现 Percolator 的完整 MVCC 存储（多版本列 + GC），
时间戳主要用于事务可见性与锁判定；把它称为"完整 MVCC"是不准确的。

5.5.4 局限

- 未处理时钟漂移超阈值的告警与降级；
- TSO 单点（有 `ResolvedTimestampService` 冗余设计，未做故障注入验证）。

5.6 基于 Percolator 思想的分布式事务协调

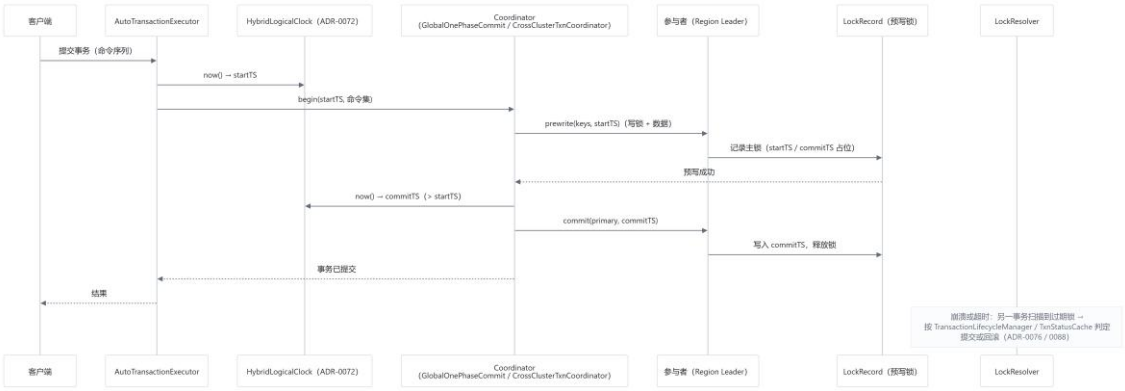


图 5-3　分布式事务协调时序（startTS/commitTS、预写锁、提交与锁清理）

5.6.1 协议

事务协议采用 Percolator 的"两阶段 + 时间戳"骨架（Peng & Dabek, OSDI 2010）[13]，其两阶段提交的基本语义见 Bernstein 等 [19]：

1. **Prewrite**：用 `startTS` 在各参与者写入数据与锁（`LockRecord`）；
2. **Commit**：协调者取 `commitTS > startTS`，先提交主锁，再提交其余锁；
3. **清理**：后台 `LockResolver` 扫描过期锁，依据事务状态决定提交或回滚。

与经典 2PC 的差异：参与者不阻塞等待协调者，而是把"锁 + 时间戳"写进存储，由后续读/后台任务解析——这正是 Percolator 用"单点时钟 + 锁表"替代阻塞式 2PC 的思路。

5.6.2 实现组件

组件	职责	证据
`GlobalOnePhaseCommit` / `CrossClusterTxnCoordinator`	协调者（本地/跨集群）	`GlobalOnePhaseCommitTest`（E2）
`LockRecord` / `TxnStatusCache`	锁与事务状态	`CrossNodeTransactionTest`（E2）
`LockResolver` / `TransactionLifecycleManager`	锁清理与生命周期	`CoordinatorCrashRecoveryTest`（E2）
`TxnHeartbeatManager` / `TxnTimeoutScheduler`	心跳与超时（ADR-0088）	`TxnNetworkFailureTest`（E2）
`PersistentExecJournal`	决策日志持久化（ADR-0081/0091）	`PersistentExecJournalTest`（E1）

5.6.3 决策顺序

主锁提交先于其余锁（ADR-0087）：只要主锁状态可查，其余锁的归属就能确定，避免"部分提交、无法判定"的中间态。该顺序由 `CrossClusterDecisionLog` 记录，崩溃后据此恢复。

5.6.4 与 Percolator 的差异（必须写明）

维度	Percolator	本项目
底层存储	Bigtable²（多版本）	自研 LSM（单版本 + 锁记录）
快照读	`readTS` 多版本可见	`readTS` 仅用于事务可见性判定（E2）
锁清理	客户端惰性清理	后台 `LockResolver` 主动清理
跨集群	不涉及	有 `CrossClusterTxnCoordinator`（E3，单机模拟）

5.6.5 局限

- 未实现 Percolator 的多版本存储与 GC；
- 跨机房/跨云事务只有单机模拟（E3）；
- 无分布式死锁检测（依赖超时与清理）。

5.7 数据迁移与一致性保障

机制	做法	证据
Region 迁移	`RegionRaftMigrationManager`：先追平日志，再切换 Leader，最后改元数据	`RegionChaosTest`（E2）
拆分期间写入	`SplitWriteBuffer` 承接增量，快照完成后回放	`RegionSplitTest`（E1）
跨分片事务	两阶段 + 时间戳（§5.6）	`CrossNodeTransactionTest`(E2)
冷热迁移	第三章的 tiering 机制，与集群层正交	`SstableInventoryScaleTest` 等（E1）

一致性边界：**同一 Region 内**由 Raft 保证日志顺序；**跨 Region** 由事务时间戳与锁保证；

两者都不提供跨 Region 的线性一致读。

5.8 测试验证与局限性分析

5.8.1 测试规模

域	测试类	代表用例
集群	62	`ChaosClusterTest`（20 例）、`FailoverTest`、`FailureInjectionTest`、`TcpClusterIntegrationTest`
Region	12	`RegionSplitTest`、`RegionMergeTest`、`RegionChaosTest`、`FiveRegionChaosTest`
事务	33	`CoordinatorCrashRecoveryTest`、`CrossNodeTransactionTest`、`TxnNetworkFailureTest`、`Phase21DistributedChaosTest`

本次全量回归 15,014 次执行、0 失败，上述测试类全部包含在内。

5.8.2 已验证的结论

1. Leader 失效后可重新选举，已提交条目不被回滚（`ChaosClusterTest`）；
2. 日志持久化后重启可恢复未应用条目（`Phase22DiskChaosTest`）；
3. Region 拆分/合并后数据范围正确、无丢失（`RegionSplitTest/RegionMergeTest`）；
4. 事务在协调者崩溃后可依据决策日志恢复（`CoordinatorCrashRecoveryTest`）；
5. 原子字符串操作经 Raft 复制后结果一致（`ReplicatedAtomicOpsTest`）。

5.8.3 未验证的结论（不得写成已实现）

表 5-2　未验证结论清单（不得写成已实现）

项	状态	原因
跨机网络分区与时钟漂移	E3	需多台物理机/可控网络，本环境不具备
跨机房/跨云事务	E3	仅单机模拟，无真实 RTT 与故障
掉电恢复	E3	需断电注入（第三章同理）
线性一致读	不做	设计上只提供 readTS 快照读
完整 MVCC + GC	未实现	仅为锁与可见性使用时间戳

5.8.4 小结

本章的分布式能力可以概括为：**在单机多进程环境下，实现了可运行的 Raft、Region 分片与

基于时间戳的两阶段事务，并通过了对应的混沌与故障测试；但跨机网络、跨机房事务与掉电场景

均未验证**。这一区分既是本章的结论，也是全文"证据可追溯"原则的体现。

本章引用文献

本章引用 [2][11][13][14][19]，完整条目见文末《参考文献》。

第六章　系统测试与实验结果分析

论文正文初稿 • 对应基线 6439ecf
图表：图 6-1（故障处理与恢复流程）
数据来源：docs/testing/phase10-e2e.md、docs/benchmark/*、本次实测输出

6.1 测试目标与测试策略

6.1.1 测试要回答的问题

第三至五章说明了"系统如何设计"，本章只回答三个问题：

1. **设计是否被实现**——每个设计点在代码里是否真的存在，能否被测试观察到；
2. **实现是否有效**——在真实运行（真实内核、真实数据库、真实浏览器）下是否产生预期行为；
3. **结论的边界在哪**——哪些结论只在本环境成立，哪些根本未被验证。

因此本章遵循三条写作原则：

- **实验结果与设计描述分开**：不拿设计目标代替实测结果；
- **性能数据必须带口径**：环境、方法、样本量、统计方式缺一不可；
- **不做能力升级**：单机多进程的 Raft 测试不写成跨机生产验证。

6.1.2 分层策略

层	目标	手段	规模
单元 / 集成	组件行为正确	JUnit 5（内核、控制面）	15,014 次执行 / 93 例
前端单元	纯逻辑正确	Vitest	20 例
界面回归	关键页面不退化	Playwright 冒烟	12 项
业务闭环	端到端可用	Node API 脚本 + 真实内核/MySQL	91 条断言
故障注入	异常路径正确	进程/数据库/令牌注入	22 条断言
性能	量化能力	基准脚本 + 采样	见 6.6

为什么以 API 级为主：业务逻辑的正确性用 API 断言最快、最稳；UI 只验证"浏览器实际表现"（终端连续会话、离线三态、权限跳转），不把七条闭环全部搬进 Playwright——那样脚本会慢到没人跑。

6.2 测试环境与配置

项	值
操作系统	Windows 11（本地基准）；Docker 29.8 / WSL2（横向对比）
CPU / 内存	20 逻辑核 / 31.6 GB
JDK	Microsoft OpenJDK 21.0.12
内核	Netty + 16 分片；WAL 段 64 MB，fsync=EVERY_SEC；compaction 阈值 8；水位 70/85/95
控制面	Spring Boot 3.5 + JPA + HikariCP（connection-timeout=5000）
数据库	MySQL 8，端口 3307，独立 datadir

	（%LOCALAPPDATA%\tieringkv-mysql）
前端	Vite 5 + Vue 3.5，端口 5173，Chromium（Playwright）
端口拓扑	内核 6379 / 测试实例 6383、6384；控制面 8080；前端 5173
浏览器	Playwright Chromium，1440×1000 / 1280×800 两种视口

6.3 核心功能测试

6.3.1 存储与分层

验证点	测试类（代表）	结论
WAL 追加 / 恢复 / 损坏丢弃	WALTest、WALDurabilityTest	重放正确，损坏记录被计数丢弃
MemTable 有序 / TTL / 并发	MemTableTest、TTLManagerTest	字节序正确，过期惰性判定
SSTable 写读 / 索引 / Bloom / CRC	SSTableTest、BloomFilterTest	读路径命中率与校验符合预期
Compaction 合并与回收	CompactionTest、SstableInventoryScaleTest	输入 429.9 MB → 输出 191.6 MB
冷热迁移与背压	TieringControllerTest、MigrationSchedulerTest	迁移成功率 100%，2 MB 配额不超限

6.3.2 集群与事务

域	测试类数	代表用例
集群	62	ChaosClusterTest（20 例）、FailoverTest、TcpClusterIntegrationTest
Region	12	RegionSplitTest、RegionMergeTest、RegionChaosTest
事务	33	CoordinatorCrashRecoveryTest、CrossNodeTransactionTest、TxnNetworkFailureTest

这些测试全部在**单机多进程**模型下运行；跨机网络分区与时钟漂移未覆盖（见 6.9）。

6.3.3 控制台

模块	验证点	结果
认证	JWT 签发、过期、篡改、角色矩阵	401/403 符合预期
实例	纳管唯一性、探测 UP/DOWN、多实例隔离	18/18 字段一致且互不相同
键空间	SCAN 游标、类型识别、HSET 单字段、TTL	与内核 EXISTS 一致
可观测性	INFO 解析、三态展示、SSTable 清单	与直连内核逐字段一致

6.4 端到端业务闭环测试

6.4.1 七条闭环与断言

表 6-1　七条业务闭环的断言与代表结果

闭环	断言数	代表结果
E2E-1 认证与权限	10	改密后旧口令 401；VIEWER 写 403 / 读审计 403；匿名 401
E2E-2 实例管理	9	探测 UP 并返回版本；不可达实例 DOWN；删除后列表不含
E2E-3 键空间	14	值/类型一致；HSET 两字段删一留一；TTL 到期后内核 EXISTS=0
E2E-4 在线终端	11	PING→PONG；错误命令 error 标记；危险命令 400；历史按实例隔离
E2E-5 备份	16	恢复后含逗号/引号的值原样还原；回放数 == keyCount；并发 200+409
E2E-6 告警	10	确定性触发 → FIRING → resolve → RESOLVED
E2E-7 审计	21	时间范围/动作过滤；CSV 真解析；三类备份审计存在
合计	91	全部通过

口径说明：91 是**断言条数**，不是测试用例数，也不是测试方法数。

一条断言对应一个可证伪的事实（如"恢复回放数 == 记录 keyCount"）。

6.4.2 断言的写法

每条断言都要求"可证伪"，典型反例与正确写法：

不合格	合格
页面能打开	离线时 6 个指标全部显示 –，且红色提示条存在
备份成功	恢复后含逗号/引号的值原样还原，且回放数 == keyCount
TTL 设置成功	到期后内核 EXISTS=0（而非控制面返回 200）
导出 CSV	每行列数与表头一致（用 RFC4180 解析器校验）

6.5 故障注入与恢复测试



图 6-1　故障处理与恢复流程（四类故障的检测、降级与恢复路径）

表 6-2　四类故障注入的实测结果

故障	注入方式	断言数	结果
F1 实例离线	指向无人监听端口	6	`live` 为空、状态 DOWN、接口 502/50201、信息不含 SQL/堆栈
F2 内核崩溃	写入 100 键 → 强制终止 → 重启	5	100/100 键存活、值无损坏、进程级计数归零
F3 数据库断连	停止专用 MySQL → 请求 → 恢复	8	5,046 ms 返回 503+50300；恢复后 108 ms 自愈
F4 Token	篡改签名 / 自签过期 Token	3	篡改→401；有效签名+过期→401；有效未过期→200
合计	—	22	全部通过，环境恢复

6.6 性能测试与分析

6.6.1 口径先于数字

指标	环境	方法	统计口径	样本
Redis 对比	同机 Docker bridge，同客户端	`redis-benchmark -r 1000000 -d 64`	服务端吞吐 + 客户端 p50	每场景 20 万请求
单连接 GET/SET	本机回环，Node RESP	串行 GET+SET	客户端侧 p50/p95/p99	每轮 4 s（约 4.6–4.9 万次）
INFO 自身	同上	连续 200 次 `INFO`	p50/p95/max	200 次
Flush / 迁移	本机 SSD，`-Xmx1g`	Phase 6 调度基准	entries/s、ops/s、端到端	100K / 200K / 1M

WAL 落盘	真实 WAL + 文件系统	10 轮采样（避开内联 force）	P99 单轮"写入→文件可见"	10 轮

必须区分的四组概念：

① 内存操作延迟（MemTable 直连） vs 网络端到端延迟（客户端观测）；
② 平均延迟 vs P99（本文一律同时给出，P99 来自直方图分桶上界）；
③ 内联 fsync（ALWAYS 或首次写入） vs 后台周期落盘（EVERY_SEC）；
④ 进程崩溃恢复（已实测） vs 掉电恢复（未验证）。

6.6.2 与 Redis 7 的对比

表 6-3 与 Redis 7 的吞吐与延迟对比（同机、同客户端、同网络；与表 3-2 为同一组实验，此处用于结果分析）

场景	Tiering-KV	Redis 7	比值
SET • c=1	10,347 rps / p50 0.079 ms	21,834 / 0.039 ms	0.47×
GET • c=1	11,148 / 0.079 ms	23,641 / 0.039 ms	0.47×
SET • c=50	59,701 / 0.431 ms	165,975 / 0.159 ms	0.36×
GET • c=50	63,512 / 0.423 ms	161,160 / 0.159 ms	0.39×

公平性：Redis 关闭 RDB/AOF，Tiering-KV 保持默认 WAL——本项目承担持久化成本，

对比对己方偏保守。结论表述为"同量级、可用"，不写"性能优于 Redis"。

6.6.3 观测开销

场景	吞吐	p99
无 INFO 流量	11,452 ops/s	0.537 ms
叠加 50 次/秒 INFO	12,061 ops/s	0.503 ms

INFO 自身 p50 0.246 ms、p95 0.509 ms、max 3.286 ms。结论：观测无可测影响（在 ≤ 8 张表的规模下）。

6.6.4 持久化边界

```
[wal-latency] 10 轮后台落盘延迟(ms)= 407.4 407.8 403.1 404.9 406.6 407.0 420.8 408.5 406.3 420.6
              最坏 = 420.8 ms
```

表 6-4 WAL 持久化边界的实测结果

实验	结果
修复前：写 100 键 → 等 1.8 s → 强制终止	重启后 99/100 丢失
修复后：同场景	100/100 存活，值无损坏
后台落盘延迟（10 轮）	403 – 421 ms（最坏 420.8 ms）
空闲 2 s 文件大小	不变（dirty 标记避免空转 fsync）

6.6.5 合并与清单成本

项	结果
一次全量合并	输入 429.9 MB → 输出 191.6 MB，1,487 ms
清单快照（1,000 张表）	52 μs；渲染 2,236 μs；输出 97,003 B

冷层精确计数（6 表 / 597 MB）	首次 17.1 ms，随后 0.63/0.39 ms

6.7 工程质量与覆盖率

表 6-5　测试与质量指标汇总

指标	结果	说明
内核测试执行	15,014（0 失败，13 跳过）	Surefire Tests run，含参数化展开
控制面测试	93（21 个类，0 失败）	JUnit 5 + MockMvc
业务闭环断言	91	7 条闭环
故障注入断言	22	4 类故障
Playwright	17/17	3 组界面场景
Vitest	20/20	格式化、排序/分页
UI 回归	12/12	页面冒烟
行覆盖率	62.57%（1,329/2,124）	控制面，未排除业务类
分支覆盖率	40.71%（265/651）	同上
指令覆盖率	59.12%（5,913/10,001）	同上

6.7.1 关于 13 项跳过

13 项跳过全部是**环境门控**，不计入通过数量：

测试类	数量	跳过原因
`KubernetesInClusterValidationTest`	3	缺少 `TIERINGKV_KIND_CLUSTER` 环境变量
`RealBlockDeviceChaosTest` / `RealBlockDeviceExerciseTest`	6	仅 Linux，Windows 上禁用
`RealContainerDiskChaosTest` / `RealNetworkChaosTest`	4	需要容器化磁盘/网络混沌环境

6.7.2 覆盖率的正确解读

- 覆盖率衡量"代码被测试触达的比例"，**不是可靠性指标**，也不能推出"缺陷更少"；
- 本项目不设及格线，覆盖率作为**可复算的基线**：任何一次 `mvn test` 都能得到同一口径；
- 统计包含 DTO / record 的访问器，因此数字偏低但真实。

6.8 缺陷发现与修复验证

6.8.1 缺陷矩阵

表 6-6　六个缺陷的发现、修复与验证

缺陷	原始问题	修复	验证方式	结果
TD-101	同实例并发备份双双成功	按实例串行化检查+落库	并发探针 5 轮	5/5 → 200/409
TD-102	失败操作审计随事务回滚	`REQUIRES_NEW`	拦截命令后查审计	记录可查
TD-103	空闲期 WAL 缓冲不落盘	后台 500 ms flush+fsync	强杀 + 重启校验	99/100 丢失 → 100/100
TD-104	备份操作无审计	三类动作写入审计	按 action 查询	均有记录
TD-105	数据库不可用返	三类连接异常映	停库→请求→恢	5046 ms → 503，

	回 500	射 503	复	108 ms 自愈
TD-106	空闲期空转 fsync	dirty 标记	空闲 2 s 观察文件	大小不变

6.8.2 案例一：TD-103（WAL 持久化）

问题背景：默认 fsync=EVERY_SEC，实现为"append 时若距上次 force ≥1 s 才落盘"。

复现条件：写入 100 个唯一键 → 等待 1.8 s（无后续写入）→ 强制终止进程 → 重启。

观察结果：重启后只恢复 1 个键。原因是数据停留在 64 KB 的 BufferedOutputStream，

空闲期没有任何 append，因此永远不触发 force。

修复机制：WALManager 启动守护调度线程，每 500 ms 检查 dirty；有未落盘写入才 flush + channel.force(true)。

验证过程：

1. 修复后同一场景 **100/100 键存活，值无损坏**（F2，5 条断言）；
2. 新增 WALDurabilityTest：空闲期写入 1.6 s 内必落盘；空闲 2 s 文件大小不变（避免空转）；
3. 10 轮采样得到落盘延迟 **403 – 421 ms**，把"最多丢多少"从定性变成定量。

实验结论：进程崩溃的丢失窗口 ≤ 一个调度周期（实测最坏 420.8 ms）。

不成立的推论：该实验**不能**证明掉电安全——未做断电注入。

6.8.3 案例二：TD-101（备份并发）

问题背景：export() 先查"是否存在 RUNNING 记录"，再插入新记录，两步不在同一临界区。

复现条件：Promise.all 同时发两个 POST /backups，连续 5 轮。

观察结果：**5/5 轮均返回 200/200**，同一实例并行跑了两个导出任务。

修复机制：按实例加锁，把"检查 + 落库"合并为一个临界区，导出仍在线程池执行（不阻塞请求）。

验证过程：并发探针 **5/5 轮 200/409**；E2E-5 断言"并发创建产生 200 + 409"；新增并发单测（两线程 + 屏障，断言恰好一次成功、一次冲突）。

实验结论：并发一致性缺陷只能靠"同时发请求 + 稳定复现"发现，单线程用例永远测不出来。

6.8.4 案例三：TD-105（数据库故障恢复）

问题背景：数据库不可用时，请求返回 500 + Could not open JPA EntityManager for transaction。

复现条件：停止专用 MySQL（非共享实例）→ 发登录请求 → 恢复 MySQL → 不重启控制面再发请求。

观察结果：

阶段	修复前	修复后
断连时	500 + 内部术语	503 + 50300「数据库暂不可用，请稍后重试」，5,046 ms 返回
恢复后	需重启控制面	不重启即可登录（108 ms）

修复机制：只对 CannotCreateTransactionException / DataAccessResourceFailureException / CannotGetJdbcConnectionException 三类异常映射 503，其余异常仍走原有兜底。

实验结论：控制面具备"依赖故障降级 + 自愈"能力；文案不泄漏 SQL/凭据/堆栈。

6.9 测试局限性分析

限制	说明	影响
跨机未验证	集群/事务测试均在单机多进程	不能声称生产级跨机共识
掉电未验证	无断电注入手段	持久化结论仅覆盖进程崩溃
恢复非原子	ADR-0365：失败不回滚	不能声称"恢复可回滚"
13 项跳过	K8s/块设备/容器网络	不计入通过率
覆盖率 62.57%	异常分支覆盖不足	不能等价为"缺陷更少"
性能口径	单机、page cache 热、无跨机网络	不能与 TiKV 等分布式系统直接对比
UI 覆盖窄	只 3 组场景 17 条断言	交互细节靠人工验收

另需说明：91 与 22 是**断言条数**，15,014 是**测试执行次数**（含参数化展开），

三者口径不同，不可相加或互相替代。

6.10 本章小结

1. **功能层面**：七条业务闭环 91 条断言全部通过，说明"从登录到备份恢复"的链路在真实环境下可用；
2. **可靠性层面**：四类故障注入 22 条断言通过，其中 TD-103 把"崩溃丢多少"从定性变为 403–421 ms 的定量结论；
3. **性能层面**：在承担 WAL 的前提下达到 Redis 的 0.36–0.47×，观测开销在 ≤8 张表时无可测影响；
4. **工程层面**：15,014 次内核执行 + 93 例控制面 + 20 例前端单测 + 12 项 UI 回归，

   控制面行覆盖率 62.57%（可复算基线）；
5. **边界**：跨机、掉电、恢复原子性、13 项环境门控测试均未覆盖，结论不外推。

这些结果共同支撑一个判断：**系统在单机范围内是可运行、可观测、可运维、可验证的**；

而"分布式生产级"所需的跨机验证，属于本论文未完成的部分，留待第七章讨论。

本章引用文献

本章为实验与结果分析，未引入新的学术文献；涉及的机制与工具见第 2–5 章及文末《参考文献》。

技术规范 S3（指标类型）、S4（语义约定）见文末"技术规范"清单。

第七章　总结与展望

论文正文初稿 • 对应基线 6439ecf

写作约束：成果不超出表 6-5 的实测指标；局限不超出表 5-2 与 6.9 的既有结论；不引入开发过程编号。

7.1 全文工作总结

本文围绕"如何用有限内存承载超出内存容量的键值数据，并让这套系统可观测、可运维、可验证"这一问题，

设计并实现了一个 Redis 协议兼容的冷热分层键值存储系统，以及与之配套的独立 Web 运维控制台。

研究起点是一个现实矛盾：纯内存实现能跑通协议却受制于堆容量，直接引入成熟存储组件又使核心机制成为黑盒。

因此本文选择自己实现完整的 LSM 写入与读取链路——预写日志、内存表、不可变磁盘表与合并，

再在其上叠加冷热分层，使"内存只保留热数据"成为结构性约束而非调参结果；

同时对冷层、缓存、迁移与合并等关键环节建立可观测指标，让系统行为可以被外部验证。

在实现路径上，系统分为数据平面与控制平面两个独立进程：数据平面负责协议解析、命令执行与存储引擎，

控制平面通过标准 RESP 协议纳管内核，提供认证授权、指标看板、键空间浏览、在线终端、告警、备份恢复与审计。

两者不共享进程、不共享存储，边界清晰，因而可以独立部署与验证。

在验证路径上，本文没有停留在"功能可用"，而是按单元与集成测试、端到端业务闭环、故障注入、性能基准、

覆盖率与静态门禁五个层次建立证据链，并对每条结论标注其适用的验证边界。

7.2 主要工作与工程成果

本文的工作可以归纳为五项，每一项都对应可复现的实验结果（见表 6-5）。

（1）实现了可解释的 LSM 存储内核。 针对"随机写转顺序写"这一目标，系统实现了分段预写日志、

分片有序内存表、带块索引与布隆过滤器的不可变磁盘表，以及按表数阈值触发的合并。

写入经日志持久化后再进入内存表，内存用量由统一记账驱动水位判定，

超过阈值时依次触发落盘、异步迁移与背压。实验表明：在承担预写日志持久化成本的前提下，

系统吞吐达到同机 Redis 的 0.36－0.47 倍；分层调度在十万至百万条目规模下迁移成功率 100%。

（2）把"冷热分层"从概念做成了可度量的机制。 内存配额、水位线、迁移与块缓存共同构成分层闭环，

冷数据落盘后在读取时按需回源。实测在 2 MB 配额下连续写入两万次，峰值内存 350 KB，未触发背压异常；

在 597 MB 冷数据、默认缓存容量下，块缓存命中率达到 84.8%，而在小缓存与大工作集场景下降到 3.2%。

后者不是缺陷，而是"缓存容量与工作集之比"的真实反映，系统选择如实展示而非平滑处理。

（3）实现了独立的控制平面。 控制平面以标准 RESP 与内核交互，不新增管理命令；

指标全部来自内核对计数器的直接读取，缺失、为零与请求失败三种情况在界面上分别呈现。

业务闭环验证覆盖认证与权限、实例管理、键空间读写、在线终端、备份恢复、告警与审计七条链路，

共 91 条可证伪断言全部通过；四类故障注入共 22 条断言，覆盖实例离线、内核崩溃、数据库断连与令牌失效。

（4）建立了以"可证伪断言"为核心的工程验证体系。 系统级回归共 15,014 次测试执行、0 失败；

控制平面 93 例测试、行覆盖率 62.57%（1,329/2,124）；前端单元测试 20 例、界面回归 12 项、

浏览器场景 17 条断言。更重要的是，这套体系在验证过程中发现了六个真实缺陷，

其中"预写日志在空闲期不落盘"导致崩溃后百键仅存活其一，"并发提交备份"导致同实例并行导出，

这些缺陷都不是靠阅读代码发现，而是靠断言逼出来的。修复后同一场景分别达到 100/100 键存活与稳定的 200/409 语义。

（5）形成了可追溯的工程文档与决策记录。 每一项设计选择（键序语义、缓存策略、合并阈值、

恢复语义、失败码划分）都有对应的决策记录与测试依据，指标口径（测试方法数、执行次数、断言条数、覆盖率定义）

在全文统一，避免把不同口径的数字混用。

7.3 系统局限性

本文严格区分"已验证"与"未验证"，以下边界同时也是结论的适用范围。

（1）分布式能力仅在单机多进程环境下验证。 系统实现了 Raft 选举、日志复制、快照与 Region 拆分合并，

并通过了集群与事务的混沌测试；但这些测试都在同一台机器的多个进程间进行，网络分区由进程或传输层模拟，

未覆盖真实网络抖动与时钟漂移。因此本文不声称具备生产级跨机共识能力。

（2）时间戳机制不等于完整的多版本并发控制。 系统用混合逻辑时钟与时间戳分配器为事务赋予开始与提交时间戳，

读请求据此做可见性判定；但没有多版本存储与垃圾回收，时间戳主要用于事务与锁的判定。

将其描述为完整 MVCC 是不准确的。

（3）快照读不等于线性一致读。 读请求返回的是某一时间戳下已提交的数据视图，

不保证读到最新写入，也不提供跨分区的全局顺序。

（4）进程崩溃恢复不等于掉电安全。 后台每 500 ms 将日志缓冲刷出并落盘，实测最坏延迟 420.8 ms，

因此进程被强制终止时最多丢失一个调度窗口内的写入；但未进行断电或文件系统级崩溃注入，

不对掉电场景作任何持久化承诺。

（5）备份恢复是合并式回放，而非原子恢复。 恢复逐条重放，失败不回滚，已写

入部分保留；

重复恢复结果一致，但两个恢复并发执行会互相覆盖。

（6）测试覆盖存在明确缺口。 13 项测试因缺少集群、真实块设备或容器化混沌环境而跳过，不计入通过；

控制平面行覆盖率 62.57%，异常分支覆盖不足；界面测试只覆盖三组关键场景。

覆盖率衡量的是代码被触达的比例，不是可靠性指标。

7.4 后续研究与改进方向

后续工作应当沿已有证据缺口推进，而不是另起方向。

（1）补齐跨机验证。 在多台物理机或可控网络环境中重复现有的集群混沌测试，

注入网络分区、延迟抖动与时钟漂移，检验选举收敛时间与提交延迟的分布。这是把"可实现"推进为"可依赖"的前提。

（2）补齐掉电与文件系统级故障注入。 在虚拟机或专用环境中模拟断电与块设备损坏，

验证预写日志的校验、截断与恢复路径，为持久化承诺提供完整证据。

（3）实现恢复的并发保护与原子语义。 为恢复操作引入互斥与临时命名空间，

使失败可以回滚、重复执行保持幂等，从而把当前的"合并式回放"升级为可回滚的恢复。

（4）明确多版本能力的目标边界。 若要支持快照隔离，需要在存储层引入多版本与垃圾回收，

并重新评估空间放大；若不实现，则应在接口文档中明确只提供时间戳可见性判定，避免使用者误解。

（5）把环境门控测试纳入持续集成。 将需要 Kubernetes、块设备与容器网络的测试放到具备相应环境的流水线中执行，

并对控制平面的异常分支补充用例，提升覆盖率的结构性短板。

7.5 本章小结

本文完成了一个"能装下超出内存的数据、能被观测、能被运维、也能被证伪"的键值存储系统：

在单机范围内，它把 LSM 主链路、冷热分层与控制平面整合为一个可运行的整体，并用分层测试与故障注入给出了

可复现的证据；在结论层面，它明确划出了尚未验证的边界。

对一个以工程能力为目标的毕业设计而言，比"功能更多"更有价值的，是知道每一项能力究竟被证明到了什么程度。

这正是本文希望留下的方法论。

参考文献

[1] O'NEIL P E, CHENG E, GAWLICK D, 等. The log-structured merge-tree (LSM-tree). Acta Informatica, 1996.33(4).
[2] CHANG F, DEAN J, GHEMAWAT S, 等. Bigtable: a distributed storage system for structured data. Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation. Berkeley: USENIX Association, 2006.
[3] QIU J, YUAN F, GAO M, 等. HotRAP: hot record retention and promotion for LSM-trees with tiered storage. Proceedings of the 2025 USENIX Annual Technical Conference. Berkeley: USENIX Association, 2025.
[4] XU P, 等. RocksMash: building a fast and efficient LSM-tree store by integrating local storage with cloud storage. ACM Transactions on Architecture and Code Optimization, 2022.19(3).
[5] DONG H,ZHANG C,LI G,等. Cloud-native databases: a survey. IEEE Transactions on Knowledge and Data Engineering, 2024.36(12).
[6] ZHANG Q, 等. Rethinking data management systems for disaggregated data centers. Proceedings of the 10th Biennial Conference on Innovative Data Systems Research. 2020.
[7] DAYAN N, ATHANASSOULIS M, IDREOS S. Monkey: optimal navigable key-value store. Proceedings of the 2017 ACM International Conference on Management of Data. 2017.
[8] DAYAN N, IDREOS S. Dostoevsky: better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. Proceedings of the 2018 ACM International Conference on Management of Data. 2018.
[9] 吕萌, 华文镝, 谢平. 基于 LSM 树的键值存储系统技术研究综述. 计算机科学, 2023.50(8).
[10] DECANDIA G, HASTORUN D, JAMPANI M, 等. Dynamo: Amazon's highly available key-value store. Proceedings of the 21st ACM Symposium on Operating Systems Principles. 2007.
[11] ONGARO D, OUSTERHOUT J. In search of an understandable consensus algorithm. Proceedings of the 2014 USENIX Annual Technical Conference. 2014.
[12] CORBETT J C, DEAN J, EPSTEIN M, 等. Spanner: Google's globally-distributed database. Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation. 2012.
[13] PENG D, DABEK F. Large-scale incremental processing using distributed transactions and notifications. Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation. 2010.
[14] KULKARNI S S, DEMIRBAS M, MADAPPA D, 等. Logical physical clocks. Proceedings of the 18th International Conference on Principles of Distributed Systems. Cham: Springer, 2014.
[15] LI B, 等. Enjoy your observability: an industrial survey of microservice tracing and analysis. Empirical Software Engineering, 2022.27(1).
[16] BLOOM B H. Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 1970.13(7).
[17] MEGIDDO N, MODHA D S. ARC: a self-tuning, low overhead replacement cache. Proceedings of the 2nd USENIX Conference on File and Storage Technologies. 2003.
[18] KLEPPMANN M. Designing data-intensive applications. Sebastopol: O'Reilly Media, 2017.
[19] BERNSTEIN P A, HADZILACOS V, GOODMAN N. Concurrency control and recovery in database systems. Reading: Addison-Wesley, 1987.
[20] GRAY J, REUTER A. Transaction processing: concepts and techniques. San Francisco: Morgan Kaufmann, 1992.
[S1] Redis. Redis protocol specification (RESP). https://redis.io/docs/latest/develop/reference/protocol-spec/.

[S2] Netty Project. Netty documentation. https://netty.io/.

致　　谢

本论文的完成离不开许多人的帮助。

首先感谢我的指导教师。从选题方向的确定、系统架构的取舍，到实验方案的设计与论文结构的调整，老师都给出了具体而耐心的指导。尤其在"哪些结论可以写、哪些还不能写"这一问题上，老师反复强调证据边界，这直接影响了本文第六章与第七章的写作方式。

感谢学院各位老师在课程与开题阶段给予的意见，使我在存储引擎与分布式系统方向上建立了必要的理论基础。

感谢同组同学在环境搭建、测试联调与论文校对中的帮助。系统验证阶段的大量重复执行与故障复现，离不开他们的配合。

感谢我的家人一直以来的理解与支持。

最后，感谢开源社区。本文的实现参考并受益于 LSM-Tree、Raft、Percolator 等公开研究成果，以及 Redis、Netty、Spring Boot 等开源项目；没有这些工作，本系统的完成是难以想象的。

由于本人水平有限，论文中难免存在不足与错误，恳请各位老师批评指正。